VOL. 02

CSS

Layout, styling and design systems for the web

Source	web.dev/learn/css
Generated	2026-10-10
Level	Beginner to intermediate
Estimated study time	30 hours
Language	English
Volume	Vol. 02 - CSS

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 10
Chapter map 10
Chapter 1 - How CSS works 11
1.1 Summary 11
1.2 Concepts 11
A rule, dissected 11
Three ways to attach CSS 11
The at-rules you will actually use 12
Cascade layers 12
1.3 Examples 12
1.4 Common mistakes and best practices 14
1.5 Practice exercises 14
Solutions 15
1.6 Mini test 15
Answer key and explanations 15
Chapter 2 - The box model 15
2.1 Summary 15
2.2 Concepts 16
The four areas 16
content-box versus border-box 16
Margin collapsing 17
Box types 17
2.3 Examples 17
2.4 Common mistakes and best practices 18
2.5 Practice exercises 19
Solutions 19
2.6 Mini test 19
Answer key and explanations 19
Chapter 3 - Selectors and combinators 20
3.1 Summary 20
3.2 Concepts 20
Simple selectors 20

Pseudo-classes worth memorising . . . 21
Pseudo-elements . . . 21
Combinators . . . 21
`:is()`, `:where()` and `:not()` . . . 22
3.3 Examples . . . 22
3.4 Common mistakes and best practices . . . 23
3.5 Practice exercises . . . 23
Solutions . . . 23
3.6 Mini test . . . 24
Answer key and explanations . . . 24
Chapter 4 - Cascade, specificity, inheritance, nesting . . . 24
4.1 Summary . . . 24
4.2 Concepts . . . 25
The cascade order . . . 25
Specificity in numbers . . . 25
Which properties inherit . . . 25
Global keywords . . . 25
Native nesting . . . 26
4.3 Examples . . . 26
4.4 Common mistakes and best practices . . . 27
4.5 Practice exercises . . . 27
Solutions . . . 28
4.6 Mini test . . . 28
Answer key and explanations . . . 28
Chapter 5 - Color, backgrounds and gradients . . . 28
5.1 Summary . . . 28
5.2 Concepts . . . 29
Colour syntaxes . . . 29
The background shorthand . . . 29
Gradient types . . . 30
Contrast . . . 30
5.3 Examples . . . 30
5.4 Common mistakes and best practices . . . 31
5.5 Practice exercises . . . 32
Solutions . . . 32

5.6 Mini test . . . 32
Answer key and explanations . . . 32
Chapter 6 - Sizing units and spacing . . . 32
6.1 Summary . . . 33
6.2 Concepts . . . 33
Unit reference . . . 33
Percentage gotcha . . . 33
Fluid values . . . 33
Intrinsic sizing . . . 34
6.3 Examples . . . 34
6.4 Common mistakes and best practices . . . 35
6.5 Practice exercises . . . 35
Solutions . . . 35
6.6 Mini test . . . 35
Answer key and explanations . . . 36
Chapter 7 - Layout, positioning and stacking . . . 36
7.1 Summary . . . 36
7.2 Concepts . . . 36
Positioning . . . 36
Stacking contexts . . . 37
Float . . . 37
7.3 Examples . . . 37
7.4 Common mistakes and best practices . . . 38
7.5 Practice exercises . . . 39
Solutions . . . 39
7.6 Mini test . . . 39
Answer key and explanations . . . 40
Chapter 8 - Flexbox . . . 40
8.1 Summary . . . 40
8.2 Concepts . . . 40
Container properties . . . 40
Item properties . . . 41
The two axes . . . 41
The most useful one-liner . . . 41
8.3 Examples . . . 41

8.4 Common mistakes and best practices . . . 43
8.5 Practice exercises . . . 43
Solutions . . . 43
8.6 Mini test . . . 44
Answer key and explanations . . . 44
Chapter 9 - Grid . . . 44
9.1 Summary . . . 44
9.2 Concepts . . . 44
Defining tracks . . . 44
Placing items . . . 45
Named areas . . . 45
Alignment . . . 45
9.3 Examples . . . 45
9.4 Common mistakes and best practices . . . 46
9.5 Practice exercises . . . 47
Solutions . . . 47
9.6 Mini test . . . 47
Answer key and explanations . . . 47
Chapter 10 - Logical properties, responsive design, container queries . . . 48
10.1 Summary . . . 48
10.2 Concepts . . . 48
Logical properties map . . . 48
Writing mode . . . 49
Media queries . . . 49
Container queries . . . 49
10.3 Examples . . . 49
10.4 Common mistakes and best practices . . . 50
10.5 Practice exercises . . . 51
Solutions . . . 51
10.6 Mini test . . . 51
Answer key and explanations . . . 51
Chapter 11 - Custom properties and functions . . . 52
11.1 Summary . . . 52
11.2 Concepts . . . 52
Defining and using . . . 52

Runtime changes . . . 52
Typed custom properties with `@property` . . . 53
Maths functions . . . 53
11.3 Examples . . . 53
11.4 Common mistakes and best practices . . . 55
11.5 Practice exercises . . . 55
Solutions . . . 55
11.6 Mini test . . . 55
Answer key and explanations . . . 56
Chapter 12 - Typography, lists, counters . . . 56
12.1 Summary . . . 56
12.2 Concepts . . . 56
A readable base . . . 56
Font stack . . . 57
Lists and markers . . . 57
Counters . . . 57
12.3 Examples . . . 57
12.4 Common mistakes and best practices . . . 59
12.5 Practice exercises . . . 59
Solutions . . . 59
12.6 Mini test . . . 59
Answer key and explanations . . . 60
Chapter 13 - Borders, shadows, focus . . . 60
13.1 Summary . . . 60
13.2 Concepts . . . 60
Border shorthand . . . 60
Shadows . . . 61
Focus . . . 61
13.3 Examples . . . 61
13.4 Common mistakes and best practices . . . 62
13.5 Practice exercises . . . 62
Solutions . . . 63
13.6 Mini test . . . 63
Answer key and explanations . . . 63
Chapter 14 - Overflow, scrolling, cursors . . . 63

14.1 Summary . . . 63
14.2 Concepts . . . 64
Overflow values . . . 64
Scroll snapping . . . 64
Smooth scrolling and anchor offsets . . . 64
Cursors . . . 65
14.3 Examples . . . 65
14.4 Common mistakes and best practices . . . 66
14.5 Practice exercises . . . 67
Solutions . . . 67
14.6 Mini test . . . 67
Answer key and explanations . . . 67
Chapter 15 - Filters, blend modes, clipping and masking . . . 68
15.1 Summary . . . 68
15.2 Concepts . . . 68
Filter functions . . . 68
Blend modes . . . 68
Clip-path shapes . . . 69
Masks . . . 69
15.3 Examples . . . 69
15.4 Common mistakes and best practices . . . 70
15.5 Practice exercises . . . 70
Solutions . . . 71
15.6 Mini test . . . 71
Answer key and explanations . . . 71
Chapter 16 - Transitions, animations, view transitions . . . 71
16.1 Summary . . . 72
16.2 Concepts . . . 72
Transitions . . . 72
Timing functions . . . 72
Animations . . . 72
Reduced motion . . . 73
View transitions . . . 73
16.3 Examples . . . 73
16.4 Common mistakes and best practices . . . 74

16.5 Practice exercises . . . 75
Solutions . . . 75
16.6 Mini test . . . 75
Answer key and explanations . . . 76
Chapter 17 - Accessibility and performance . . . 76
17.1 Summary . . . 76
17.2 Concepts . . . 76
Contrast and states . . . 76
Forced colours and contrast preferences . . . 77
Deferred rendering . . . 77
Critical CSS . . . 77
17.3 Examples . . . 77
17.4 Common mistakes and best practices . . . 78
17.5 Practice exercises . . . 79
Solutions . . . 79
17.6 Mini test . . . 79
Answer key and explanations . . . 79
Cheat sheet . . . 80
18.1 Syntax . . . 80
18.2 At-rules . . . 80
18.3 Selectors . . . 80
18.4 Specificity and cascade . . . 80
18.5 Box model . . . 80
18.6 Display . . . 81
18.7 Positioning . . . 81
18.8 Flexbox . . . 81
18.9 Grid . . . 81
18.10 Sizing . . . 81
18.11 Logical properties . . . 81
18.12 Colour . . . 81
18.13 Backgrounds . . . 81
18.14 Typography . . . 81
18.15 Lists . . . 81
18.16 Borders and shadows . . . 81

18.17 Overflow and scroll 82
18.18 Effects 82
18.19 Motion 82
18.20 Accessibility and performance 82
18.21 The values you type most 82
Final test 82
19.1 Final test answers 83
Sources 84

How to use this volume

Prerequisites: HTML (Vol. 01). You do not need to be an HTML expert, but you must be comfortable with elements, attributes, classes and the document skeleton, because CSS is written to select and style those elements.

Estimated study time: about 30 hours. CSS is larger than it looks: the syntax is small, but layout, specificity and responsive design take time to internalise.

How the chapters work: every chapter has a summary, the concepts explained plainly, runnable examples with the expected result, common mistakes with best practices, practice exercises with solutions, and a mini test with an answer key.

A rule for this volume: open your browser's DevTools and inspect the computed styles of every example. CSS is a language you read with developer tools more than with your eyes.

Chapter map

#	Chapter	You will learn
1	How CSS works	syntax, inclusion, at-rules, cascade basics
2	The box model	content, padding, border, margin
3	Selectors and combinators	how to target anything
4	Cascade, specificity, inheritance, nesting	which rule wins
5	Color, backgrounds and gradients	painting boxes
6	Sizing units and spacing	px, rem, %, vw, ch, gap
7	Layout, positioning, stacking	flow, position, z-index
8	Flexbox	one-dimensional layout
9	Grid	two-dimensional layout
10	Logical properties, responsive design, container queries	adaptive layouts
11	Custom properties and functions	variables, calc, clamp
12	Typography, lists, counters	readable text
13	Borders, shadows, focus	depth and affordance
14	Overflow, scrolling, cursors	content that does not fit
15	Filters, blend modes, clipping, masking	visual effects
16	Transitions, animations, view transitions	motion with purpose
17	Accessibility and performance	contrast, reduced motion, budgets

#	Chapter	You will learn
18	Cheat sheet	one-page reference
19	Final test	30 questions with answers

Chapter 1 - How CSS works

1.1 Summary

- CSS is a list of rules; each rule has a selector and a declaration block.
- A declaration is a property, a colon, a value and a semicolon.
- Styles are attached with a `<link>` in the head, a `<style>` element, or an inline `style` attribute.
- At-rules start with `@`: `@media`, `@import`, `@supports`, `@layer`, `@font-face`, `@keyframes`, `@container`.
- The cascade decides which declaration wins: origin, layers, specificity, order.
- CSS is forgiving: an unknown property or invalid value is ignored, not fatal.
- Comments are `/* ... */`; there is no `//` comment in plain CSS.
- One stylesheet, cached once, styles the whole site.

1.2 Concepts

A rule, dissected

CSS

```css
/* selector      declaration block */
.card {
  color: #1f2328;      /* property: value */
  padding: 1rem;
}
```

Part	Name
`.card`	selector (which elements)
`color`, `padding`	properties
`#1f2328`, `1rem`	values
`{ ... }`	declaration block

Three ways to attach CSS

HTML

```html
<!-- 1. external stylesheet: the default choice -->
<link rel="stylesheet" href="/assets/main.css">

<!-- 2. a style element in the document -->
<style>
  .page-header { border-bottom: 1px solid #d8dee4; }
</style>

<!-- 3. inline: avoid, it cannot be overridden cleanly -->
<p style="color: red">Only this paragraph.</p>
```

Prefer the external stylesheet: it is cached, shared by every page, and can be minified. Use `<style>` for critical above-the-fold rules or for styles that must live with a component. Avoid `style` attributes except for values set by JavaScript.

The at-rules you will actually use

At-rule	Purpose
`@import url("...")`	load another stylesheet (slower than `<link>`)
`@media (min-width: 40em) { }`	conditional rules
`@supports (display: grid) { }`	feature detection
`@layer base, components;`	control cascade layers
`@font-face { }`	define a custom font
`@keyframes name { }`	an animation
`@container (min-width: 30em) { }`	query a parent's size

Cascade layers

css

```css
@layer reset, base, components, utilities;

@layer reset {
  * { margin: 0; }
}

@layer components {
  .button { padding: 0.5rem 1rem; }
}

@layer utilities {
  .mt-4 { margin-top: 1rem; }
}
```

Rules in a later layer beat earlier ones regardless of specificity, which is how you stop specificity wars.

1.3 Examples

Example 1: a complete tiny stylesheet.

CSS

```css
/* main.css */
:root {
  --ink: #1f2328;
  --accent: #0969da;
}

body {
  font: 16px/1.5 system-ui, sans-serif;
  color: var(--ink);
}

a {
  color: var(--accent);
  text-decoration: underline;
}

.card {
  padding: 1rem;
  border: 1px solid #d8dee4;
  border-radius: 8px;
}
```

Expected result: dark body text on a white background, blue underlined links, and `.card` boxes with a light border, rounded corners and inner padding.

Example 2: a media query.

CSS

```css
.cards {
  display: grid;
  gap: 1rem;
}

@media (min-width: 40em) {
  .cards {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

Expected result: a single column on narrow screens, three equal columns from `40em` (about 640px) upwards.

Example 3: invalid rules are skipped, not fatal.

CSS

```css
/* BAD EXAMPLE on purpose: the first property is misspelled */
.box {
  colour: red;          /* misspelled: ignored */
  color: blue;          /* this one applies */
  padding: 1rem 2rem;   /* valid */
  width: -10px;         /* invalid: ignored */
}
```

The element is blue, padded, and has no fixed width. Unknown properties and invalid values are dropped at parse time, which is why CSS "fails soft".

Example 4: a style element in the document.

HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Inline styles demo</title>
  <style>
    .lead { font-size: 1.25rem; color: #57606a; }
  </style>
</head>
<body>
  <p class="lead">A slightly larger, muted introduction paragraph.</p>
</body>
</html>
```

Expected result: the paragraph is rendered larger and grey.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Inline `style` everywhere	cannot be reused or overridden	classes in a stylesheet
Missing semicolons	the next declaration is swallowed	always terminate with `;`
`* { margin: 0 }` blanket resets	kills useful spacing, hard to debug	scoped reset or a layer
Using `!important` to win fights	the cascade stops working	fix the selector instead
Massive single stylesheet	hard to maintain	split by concern, keep the request count low
Deeply nested selectors	specificity inflation	keep selectors flat, use layers
Styling elements instead of states	fragile	style classes and states

- Validate with the W3C CSS validator while learning; it explains every mistake.
- Keep the number of stylesheets small: one cached file is faster than ten.

1.5 Practice exercises

1. Write a rule that makes all `blockquote` elements italic with a left border.
2. Attach `main.css` to a document with a `<link>`.
3. What happens to a declaration that ends without a semicolon when another declaration follows?
4. Write a media query that applies `font-size: 14px` below `480px`.
5. Why is `@layer` useful on a large project?

Solutions

1. `css blockquote { font-style: italic; border-left: 3px solid #d8dee4; padding-left: 1rem; }`
2. `html <link rel="stylesheet" href="main.css">` inside `<head>`.
3. The parser recovers at the next `;`, so the following declaration is usually swallowed: the rule loses part of its effect. Always terminate declarations.
4. `css @media (max-width: 480px) { body { font-size: 14px; } }`
5. Layers give explicit ordering, so a utility class can beat a component class without `!important` and without inflating specificity, and teams can ship rules without breaking each other.

1.6 Mini test

1. Multiple choice: which character separates a property from its value? - a) `=` - b) `:` - c) `;` - d) `,`
2. True or false: `// comment` is valid CSS.
3. Multiple choice: where should a site's main stylesheet be attached? - a) in the footer - b) in `<head>` with `<link rel="stylesheet">` - c) as an inline attribute on `<html>` - d) in a `<script>`
4. Short code question: write a comment.
5. Multiple choice: what happens when a CSS value is invalid? - a) the whole stylesheet fails - b) the declaration is ignored - c) the browser shows an error page - d) the page is unstyled
6. True or false: `@media` queries test the viewport by default and a container when used with `container-type`.
7. Short answer: name three at-rules and what they do.

Answer key and explanations

1. b - `color: red`.
2. False - CSS comments are `/* ... */`.
3. b - in the head, so it loads before the first paint.
4. `/* comment */`
5. b - the invalid declaration is dropped; the rest of the rule survives.
6. True - `@media` measures the viewport, `@container` measures the nearest ancestor with `container-type`.
7. Any three of: `@media` conditional rules, `@supports` feature detection, `@layer` cascade layers, `@font-face` custom fonts, `@keyframes` animations, `@container` container queries, `@import` including another sheet.

Chapter 2 - The box model

2.1 Summary

- Every element is a rectangular box: content, padding, border, margin.
- `box-sizing: border-box` includes padding and border in the declared size.
- `width` and `height` apply to the content box unless `box-sizing` changes it.
- Margins of adjacent siblings collapse to the larger of the two.
- `min-width`/`max-width` and `min-height`/`max-height` constrain sizes.
- Percentage padding and margin resolve against the inline (width) axis.
- `display` changes the box type: block, inline, inline-block, none, flex, grid.
- `overflow` decides what happens when content does not fit.
- The box model is the single most important layout concept in CSS.

2.2 Concepts

The four areas

output

```
+-----------------------------------+
|             margin                |
|   +---------------------------+   |
|   |          border           |   |
|   |   +-------------------+   |   |
|   |   |      padding      |   |   |
|   |   |   +-----------+   |   |   |
|   |   |   |  content  |   |   |   |
|   |   |   +-----------+   |   |   |
|   |   +-------------------+   |   |
|   +---------------------------+   |
+-----------------------------------+
```

CSS

```css
.box {
  width: 200px;          /* content width */
  padding: 20px;         /* inside the border */
  border: 4px solid;     /* the frame */
  margin: 10px;          /* outside the border */
  box-sizing: border-box;/* width now means border-box */
}
```

Property	Affects	Background
content	the element's own size	painted
padding	inside space	painted (background extends under it)
border	the frame	painted by the border colour
margin	outside space	never painted

content-box versus border-box

With `content-box` (the legacy default), a `width: 200px` box with `padding: 20px` and `border: 4px` occupies `200 + 40 + 8 = 248px`.

With `border-box`, the same box occupies exactly `200px`, and the content area shrinks to `200 - 40 - 8 = 152px`.

CSS

```css
/* the pragmatic reset used by nearly every project */
*, *::before, *::after {
  box-sizing: border-box;
}
```

Margin collapsing

output

```
<div>  margin-bottom: 20px
<div>  margin-top: 30px
```

The gap is 30px, not 50px: adjacent vertical margins collapse to the larger value. Horizontal margins never collapse. Collapsing does not happen inside flex and grid containers.

Box types

display	**Behaviour**
`block`	full width, own line, respects all box properties
`inline`	flows with text, ignores width/height and vertical padding
`inline-block`	flows with text, but respects width/height
`none`	removed from layout entirely
`flex`	block-level flex container
`inline-flex`	inline-level flex container
`grid`	block-level grid container
`inline-grid`	inline-level grid container

2.3 Examples

Example 1: the two box-sizing models side by side.

CSS

```css
.a, .b {
  width: 200px;
  padding: 20px;
  border: 4px solid #0969da;
}
.a { box-sizing: content-box; }
.b { box-sizing: border-box; }
```

HTML

```html
<p class="a">content-box</p>
<p class="b">border-box</p>
```

Expected result: `.a` renders 248px wide, `.b` renders exactly 200px wide with narrower text inside.

Example 2: margin collapsing.

CSS

```css
.one { margin-bottom: 24px; }
.two { margin-top: 12px; }
```

HTML

```html
<div class="one">one</div>
<div class="two">two</div>
```

Expected result: a 24px gap, not 36px.

Example 3: preventing collapsing with flexbox.

CSS

```css
.stack {
  display: flex;
  flex-direction: column;
  gap: 12px;
}
```

Inside a flex container, child margins do not collapse, and `gap` is the modern way to space items anyway.

Example 4: a card built from the box model.

CSS

```css
.card {
  box-sizing: border-box;
  width: 320px;
  padding: 16px 20px;
  border: 1px solid #d8dee4;
  border-radius: 12px;
  margin: 24px auto;
  background: #ffffff;
}
```

Expected result: a 320px card, centred, with 16px vertical and 20px horizontal inner padding and a thin border.

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Forgetting `box-sizing`	maths you cannot trust	`border-box` reset
Percentage padding surprises	resolves against the width	use aspect-ratio or absolute units
Relying on margin collapsing	breaks inside flex/grid	use `gap`

Mistake	Why it hurts	Best practice
Vertical margins on inline elements	ignored	use `inline-block` or `display: block`
Setting fixed heights	content overflows	use `min-height`
Huge margins on `body` only	inconsistent rhythm	a small normalise or reset layer

- Use `min-height`, never `height`, on text containers.
- Prefer `gap` over margins between siblings.

2.5 Practice exercises

1. With `border-box`, what is the content width of a box with `width: 300px; padding: 10px; border: 2px solid`?
2. Two siblings have `margin-bottom: 16px` and `margin-top: 8px`. What is the gap?
3. Write the universal `border-box` reset.
4. Why does `height: 100px` on a paragraph with more text hurt?
5. Which `display` keeps an element on the text line but lets it have a width?

Solutions

1. `300 - 2*10 - 2*2 = 276px`.
2. `16px` (the larger margin wins).
3. `css *, *::before, *::after { box-sizing: border-box; }`
4. Text overflows the fixed height: the box either clips or overlaps the next element. Use `min-height` so the box can grow.
5. `inline-block`.

2.6 Mini test

1. Multiple choice: which `box-sizing` includes padding and border in the width? - a) `content-box` - b) `border-box` - c) `padding-box` - d) `margin-box`
2. True or false: adjacent vertical margins collapse to their sum.
3. Multiple choice: which property adds space inside the border? - a) `margin` - b) `padding` - c) `border-spacing` - d) `gap`
4. Short code question: give a box a 2px solid red border and 12px padding.
5. Multiple choice: which value of `display` removes an element from the layout? - a) `hidden` - b) `none` - c) `collapse` - d) `inline`
6. True or false: percentage `padding-top` resolves against the element's own width.
7. Short answer: which two properties add space between flex items?

Answer key and explanations

1. b - `border-box`.
2. False - they collapse to the larger value.
3. b - `padding`.
4. `border: 2px solid red; padding: 12px;`
5. b - `none`.
6. True - all percentage paddings and margins resolve against the inline size, which is why `padding-top: 50%` makes a square of a 2:1 box.
7. `gap` (and `row-gap`/`column-gap`).

Chapter 3 - Selectors and combinators

3.1 Summary

- A selector chooses the elements a rule applies to.
- Simple selectors: type, class, id, universal, attribute.
- Pseudo-classes match state: `:hover`, `:focus-visible`, `:nth-child()`.
- Pseudo-elements create virtual sub-elements: `::before`, `::after`, `::marker`, `::selection`.
- Combinators join selectors: descendant (space), child (`>`), next (`+`), subsequent (`~`).
- Selector lists are comma-separated and share a declaration block.
- Class selectors are the workhorse; id selectors are heavy and unreusable.
- `:is()`, `:where()` and `:not()` keep complex selectors readable.
- Specificity comes from selectors, so choose them deliberately.

3.2 Concepts

Simple selectors

Selector	Matches
`p`	all `<p>`
`.card`	elements with `class="card"`
`#hero`	the element with `id="hero"`
`*`	every element
`[type="email"]`	attribute equals
`[href^="https"]`	attribute starts with
`[href$=".pdf"]`	attribute ends with
`[class*="btn"]`	attribute contains
`[lang="en"]`	attribute is or starts with `en-`

Selector	Matches
`a:not([href])`	negation

Pseudo-classes worth memorising

Pseudo-class	Matches
`:hover`	pointer over the element
`:focus-visible`	keyboard focus (the good one)
`:focus-within`	a descendant has focus
`:first-child` / `:last-child`	position among siblings
`:nth-child(2n)` / `:nth-child(odd)`	arithmetic position
`:nth-of-type(3)`	position among same-type siblings
`:checked`	checked checkbox or radio
`:disabled` / `:read-only`	form state
`:required` / `:invalid` / `:valid`	validation state
`:placeholder-shown`	the placeholder is visible
`:has(> img)`	a parent that contains something (relational)
`:root`	the `<html>` element

Pseudo-elements

Pseudo-element	Creates
`::before` / `::after`	a generated box, needs `content`
`::first-line` / `::first-letter`	typographic sub-parts
`::marker`	the list marker or `<summary>` marker
`::placeholder`	the input placeholder
`::selection`	the highlighted text
`::backdrop`	the space behind a dialog
`::file-selector-button`	the file input button

Combinators

CSS

```css
/* descendant: any .title inside .card */
.card .title { color: var(--ink); }

/* child: .title directly inside .card */
.card > .title { font-weight: 600; }

/* next sibling: the <p> right after an <h2> */
h2 + p { margin-top: 0; }

/* subsequent siblings: every <p> after an <h2> */
h2 ~ p { color: var(--muted); }
```

`:is()`, `:where()` and `:not()`

CSS

```css
:is(h1, h2, h3) { line-height: 1.2; }        /* same as h1, h2, h3 */
:where(h1, h2, h3) { line-height: 1.2; }    /* zero specificity */
.card :not(.disabled) { opacity: 1; }
article:has(> img) { grid-column: span 2; }
```

`:where()` contributes zero specificity, which makes it ideal for defaults you want themes to override.

3.3 Examples

Example 1: attribute selectors in practice.

CSS

```css
a[href^="http"]::after {
  content: " (external)";
  font-size: 0.8em;
  color: #57606a;
}
```

Expected result: external links get a muted " (external)" hint after them, while `#fragment` links do not.

Example 2: zebra striping with `nth-child`.

CSS

```css
tbody tr:nth-child(odd) {
  background: #f6f8fa;
}
```

Expected result: alternating grey rows, without extra classes in the HTML.

Example 3: `:has()` for a parent that contains something.

CSS

```css
.card:has(img) {
  display: grid;
  grid-template-columns: 1fr 2fr;
  gap: 1rem;
}
```

Expected result: only cards that actually contain an image get the two-column layout.

Example 4: `::before` for a decorative quote mark.

CSS

```css
blockquote::before {
  content: "\201C";
  font-size: 3rem;
  line-height: 0.5;
  color: #d8dee4;
}
```

Expected result: a large light-grey opening quotation mark above the quote text (generated content, not present in the HTML).

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`#id` styling everywhere	specificity wall, no reuse	classes
Chained descendant selectors 5 deep	fragile, slow to reason about	one or two levels max
`!important` fights	unmaintainable	fix specificity or use `@layer`
`::before` without `content`	nothing renders	always set `content`
`:hover` only, no `:focus-visible`	keyboard users lose state	pair them
Styling `.btn-primary` and `.btn`	order-dependent	design tokens + layers
`div div div` for structure	coupling to markup	semantic classes

- Style classes and states, not tag soup.
- Use `:has()` to replace "add a class with JavaScript" patterns.

3.5 Practice exercises

1. Select every `input` whose `type` is `checkbox`.
2. Select the first `<li>` in every list.
3. Add a `\2192` after every internal link (`href^="/"`).
4. Why is `#header .nav a` a worse selector than `.nav a`?
5. Select `label` elements that directly follow an `input`.

Solutions

1. `input[type="checkbox"] { }`
2. `li:first-child { }`
3. `css a[href^="/"]::after { content: "\2192"; }`

4. `#header` has id specificity, so any attempt to override that rule needs an id or `!important`. The class version stays easy to override and is reusable across headers.
5. `input + label { }`

3.6 Mini test

1. Multiple choice: which selector has the highest specificity? - a) `.card` - b) `#hero` - c) `p` - d) `[type="text"]`
2. True or false: `::before` requires a `content` property to render.
3. Multiple choice: which combinator selects a direct child? - a) (space) - b) `>` - c) `+` - d) `~`
4. Short code question: select links whose `href` ends in `.pdf`.
5. Multiple choice: which pseudo-class matches keyboard focus? - a) `:focus` - b) `:focus-visible` - c) `:active` - d) `:target`
6. True or false: `:where()` adds zero specificity.
7. Short answer: what does `a:not([href])` select?

Answer key and explanations

1. b - ids have the highest specificity.
2. True - without `content` the pseudo-element is not generated.
3. b - `>`.
4. `a[href$=".pdf"]`
5. b - `:focus-visible` matches keyboard and other "text-caret" focus.
6. True - that is the difference between `:is()` and `:where()`.
7. Anchors with no `href` attribute: the placeholder anchors that are not links.

Chapter 4 - Cascade, specificity, inheritance, nesting

4.1 Summary

- The cascade resolves conflicts by origin, then layers, then specificity, then order.
- Specificity: id (100) > class/attribute/pseudo-class (10) > type (1).
- `!important` inverts the cascade: avoid it.
- Inherited properties (colour, font, line-height, visibility) flow down.
- Non-inherited properties (margin, padding, border, background) do not.
- `inherit`, `initial`, `unset` and `revert` force values.
- `@layer` gives explicit, ordered control.
- Native CSS nesting works in all modern browsers.
- Universal selectors and `*` have zero specificity but still count as a rule.

4.2 Concepts

The cascade order

output

```
1. origin and importance
   (user-agent < user < author, !important reverses)
2. @layer order (later layers win)
3. specificity (id > class > type)
4. document order (later wins)
```

Specificity in numbers

Selector	Specificity
`*`	0-0-0
`p`	0-0-1
`.card`	0-1-0
`[type="text"]`	0-1-0
`:hover`	0-1-0
`#hero`	1-0-0
`.card p a:hover`	0-3-1

Counted as (id, class/attribute/pseudo-class, type/pseudo-element).

Which properties inherit

Inherited	Not inherited
`color`, `font*`, `letter-spacing`, `line-height`	`margin`, `padding`, `border`
`text-align`, `text-indent`, `white-space`	`background*`, `width`, `height`
`visibility`, `cursor`, `list-style*`	`display`, `position`, `overflow`
`border-collapse`, `border-spacing`	`z-index`

CSS

```css
p {
  color: inherit;   /* take the parent's colour even though p is not a link */
}
```

Global keywords

Keyword	Effect
`inherit`	take the parent's computed value

Keyword	Effect
`initial`	the property's initial value
`unset`	`inherit` for inherited properties, `initial` otherwise
`revert`	roll back to the previous cascade origin (user agent)
`revert-layer`	roll back to the previous cascade layer

Native nesting

CSS

```css
.card {
  border: 1px solid #d8dee4;

  & .title {
    font-size: 1.25rem;
  }

  &:hover {
    border-color: #0969da;
  }

  @media (min-width: 40em) {
    display: grid;
  }
}
```

`&` refers to the parent selector. Nested rules are equivalent to their expanded form, and native nesting removes the need for a preprocessor.

4.3 Examples

Example 1: the same element, three rules, one winner.

CSS

```css
p { color: blue; }            /* specificity 0-0-1 */
.text { color: green; }       /* specificity 0-1-0 */
#intro { color: red; }        /* specificity 1-0-0 */
```

HTML

```html
<p id="intro" class="text">Which colour?</p>
```

Expected result: red. The id wins on specificity even though it is declared first.

Example 2: order breaking a tie.

CSS

```css
.btn { background: blue; }
.btn { background: green; }
```

Expected result: green. Equal specificity, later declaration wins.

Example 3: layers instead of `!important`.

CSS

```css
@layer base, components, utilities;

@layer components {
  .card { padding: 1rem; }
}

@layer utilities {
  .p-0 { padding: 0; }
}
```

HTML

```html
<div class="card p-0">Utility wins without !important.</div>
```

Expected result: zero padding, because `utilities` is declared after `components`.

Example 4: forcing an inherited property.

CSS

```css
button {
  font: inherit;          /* buttons do not inherit by default */
  color: currentColor;
}
```

Expected result: form controls adopt the surrounding font and colour instead of the user-agent default.

4.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`!important` everywhere	the cascade is gone	layers, fix specificity
Specificity escalation	each fix needs a stronger selector	flat class selectors
Styling `ul` when you mean `.nav`	leaks to unrelated lists	scope with classes
Inconsistent `* { }` resets	surprising side effects	a documented reset layer
Not knowing what inherits	duplicated declarations	set the shared values on `body`
Fighting specificity with inline styles	unoverridable	classes and layers

- Keep specificity at 0-1-0 or 0-2-0 and everything stays easy to override.
- When a rule cannot be overridden, ask which selector is winning.

4.5 Practice exercises

1. Order these by specificity: `.a .b`, `#x`, `p`, `p.a`.
2. Write a `@layer` declaration that puts utilities last.

3. Which value of `color` takes the parent's colour?
4. A `button` does not adopt the page font. Fix it in one declaration.
5. Two rules tie on specificity. Which wins?

Solutions

1. `#x` (1-0-0) > `.a .b` (0-2-0) > `p.a` (0-1-1) > `p` (0-0-1).
2. `@layer reset, base, components, utilities;`
3. `inherit` (or `unset` for an inherited property like `color`).
4. `font: inherit;`
5. The one declared later in the stylesheet.

4.6 Mini test

1. Multiple choice: which has specificity 0-1-0? - a) `#main` - b) `.item` - c) `p` - d) `p:hover span`
2. True or false: `!important` declarations always win over everything.
3. Multiple choice: which keyword takes the parent's computed value? - a) `initial` - b) `inherit` - c) `unset` - d) `revert`
4. Short code question: make every `button` inherit the page font.
5. Multiple choice: which property does not inherit by default? - a) `color` - b) `line-height` - c) `padding` - d) `font-family`
6. True or false: nested CSS requires a build step today.
7. Short answer: in one sentence, what does `@layer` solve?

Answer key and explanations

1. b - `.item` is one class.
2. False - a later `!important` with higher specificity still beats it, and the user's `!important` beats the author's.
3. b - `inherit`.
4. `button { font: inherit; }`
5. c - `padding` is not inherited.
6. False - native nesting is supported in all current browsers; the `&` selector makes it explicit.
7. It gives explicit ordering between groups of rules, so utilities can beat components without `!important` or specificity escalation.

Chapter 5 - Color, backgrounds and gradients

5.1 Summary

- Colour keywords, hex, `rgb()`, `hsl()`, `oklch()` and `color-mix()` all work.
- `currentColor` means "the element's own color value".
- `background` is a shorthand for eight longhand properties.
- `background-size: cover | contain` controls image scaling.
- `background-attachment`, `clip` and `origin` handle fine details.
- Gradients are images: `linear-gradient()` and `radial-gradient()`.
- `border-image` and `mask` produce effects without extra DOM.
- Contrast is a legal requirement: 4.5:1 for normal text.
- Never encode meaning in colour alone.

5.2 Concepts

Colour syntaxes

Syntax	Example
keyword	`rebeccapurple`, `transparent`
hex	`#0969da`, `#0969dacc` (alpha)
`rgb()` / `rgba()`	`rgb(9 105 218 / 0.5)`
`hsl()` / `hsla()`	`hsl(212 92% 45%)`
`oklch()`	`oklch(55% 0.2 255)` - perceptually uniform
`color-mix()`	`color-mix(in oklab, blue, white 20%)`
`light-dark()`	`light-dark(#fff, #111)`

Modern space-separated syntax is preferred; commas are legacy but still valid.

The background shorthand

CSS

```css
background:
  url("hero.jpg")        /* image */
  center / cover no-repeat   /* position / size repeat */
  #f6f8fa;               /* colour */
```

Longhand	Controls
`background-color`	the fill colour
`background-image`	gradients or `url()`
`background-position`	where the image sits
`background-size`	`auto`, `cover`, `contain`, lengths
`background-repeat`	`repeat`, `no-repeat`, `repeat-x`

Longhand	Controls
`background-attachment`	`scroll`, `fixed`, `local`
`background-origin`	padding-box, border-box, content-box
`background-clip`	which box the background paints in
`background-blend-mode`	blend image and colour

Gradient types

CSS

```css
linear-gradient(to right, #0969da, #8250df);       /* direction */
linear-gradient(180deg, rgb(0 0 0 / 0.4), transparent);
radial-gradient(circle at 20% 20%, #fff, #0969da);
conic-gradient(from 45deg, red, yellow, lime);
repeating-linear-gradient(90deg, #000 0 10px, #fff 10px 20px);
```

Gradients take the place of an image, so they scale without artefacts.

Contrast

CSS

```css
:root {
  --bg: #ffffff;
  --ink: #1f2328;   /* 15.9:1 on white */
  --muted: #57606a; /* 7.0:1 */
  --accent: #0969da;/* 4.6:1 */
}
```

5.3 Examples

Example 1: a hero with a gradient overlay.

CSS

```css
.hero {
  min-height: 60vh;
  display: grid;
  place-items: center;
  background:
    linear-gradient(rgb(0 0 0 / 0.5), rgb(0 0 0 / 0.2)),
    url("hero.jpg") center / cover no-repeat,
    #101418;
  color: #ffffff;
}
```

Expected result: a full-width hero whose photo is scaled to cover, darken by a gradient so white text stays readable.

Example 2: `currentColor`.

CSS

```css
.icon-button {
  color: #0969da;
  border: 1px solid currentColor;
  background: color-mix(in oklab, currentColor, white 90%);
}
.icon-button:hover {
  color: #8250df;   /* border and background follow automatically */
}
```

Expected result: changing `color` recolours the border and the tinted background without touching them.

Example 3: a card surface using `color-mix`.

CSS

```css
.card {
  background: color-mix(in oklab, canvas, canvastext 4%);
  border: 1px solid color-mix(in oklab, canvastext 15%, transparent);
}
```

System colour keywords (`canvas`, `canvastext`) follow the user's theme, and `color-mix` derives borders from them.

Example 4: text with a clipped gradient.

CSS

```css
.gradient-text {
  background: linear-gradient(90deg, #0969da, #8250df);
  -webkit-background-clip: text;
  background-clip: text;
  color: transparent;
}
```

Expected result: the headline is filled with a blue-to-purple gradient. Text selection becomes invisible, so use it with care.

5.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Background images with no colour fallback	flash of blank space	a matching `background-color`
`cover` on tiny devices	crops the subject	media-query the image
Pure black on pure white	harsh, and not a design system	`#1f2328` on `#ffffff`
Colour as the only signal	inaccessible	icon + text + colour
Huge background JPEGs	slow	compress, modern formats
`transparent` text for "effects"	selection is invisible	sparingly

- Define palette tokens once, in `:root` or a layer, and reference them.
- Check contrast with a tool before shipping.

5.5 Practice exercises

1. Write a `linear-gradient` from red to blue going down.
2. Scale a background image to cover its element without repeating.
3. Which property makes an element's border take its text colour?
4. Give a card a tinted background from `currentColor`.
5. Why must a gradient overlay be behind text that is white?

Solutions

1. `linear-gradient(red, blue);`
2. `background: url("x.jpg") center / cover no-repeat;`
3. `border-color: currentColor;`
4. `background: color-mix(in oklab, currentColor, white 92%);`
5. Because a photo can be bright anywhere, and white text over a bright area falls below the contrast minimum. The gradient darkens the area under the text.

5.6 Mini test

1. Multiple choice: which function mixes two colours? - a) `calc()` - b) `color-mix()` - c) `clamp()` - d) `var()`
2. True or false: a gradient is a CSS image, not an image file.
3. Multiple choice: which value scales an image to fill, cropping if needed? - a) `contain` - b) `cover` - c) `fill` - d) `none`
4. Short code question: centre a background image once, no repeat.
5. Multiple choice: what does `currentColor` refer to? - a) the user's accent colour - b) the computed value of `color` - c) the background colour - d) `#000`
6. True or false: text contrast of 3:1 is enough for normal-size body text.
7. Short answer: which colour space is perceptually uniform?

Answer key and explanations

1. b - `color-mix()`.
2. True - it is generated by the browser, so it scales freely.
3. b - `cover` fills and crops; `contain` fits inside.
4. `background: url("x.jpg") center no-repeat;`
5. b - the element's own computed `color`.
6. False - 3:1 is only for large text; normal text needs 4.5:1.
7. `oklch()` (and `oklab`) - unlike HSL, equal lightness steps look equally light.

Chapter 6 - Sizing units and spacing

6.1 Summary

- Absolute units: `px`, `pt`, `cm`. Relative: `em`, `rem`, `%`, `vw`, `vh`, `vmin`, `vmax`, `ch`, `ex`.
- `rem` is relative to the root font size; `em` to the element's own.
- `%` resolves against a different reference depending on the property.
- `min-content`, `max-content`, `fit-content` size to the content.
- `clamp()`, `min()`, `max()` and `calc()` compute values at runtime.
- `aspect-ratio` sizes boxes from a ratio.
- Use `rem` for type and spacing, `ch` for line length, `%` for fluid grids.
- `min-width: 0` and `min-height: 0` fix flex and grid overflow bugs.
- Spacing scales better from a token than from magic numbers.

6.2 Concepts

Unit reference

Unit	Relative to
`px`	absolute (CSS pixel)
`em`	the element's font size (or parent for `font-size` itself)
`rem`	the root element font size
`%`	the containing block (or the font size for `line-height`)
`vw` / `vh`	1% of viewport width / height
`vmin` / `vmax`	the smaller / larger of vw and vh
`ch`	the width of the character `0`
`cap`, `ic`, `lh`	cap height, ideograph, line height

Percentage gotcha

CSS

```css
.child {
  width: 50%;        /* 50% of the parent's content width */
  padding-top: 50%;  /* 50% of the parent's WIDTH, not its height */
}
```

That is why `padding-top: 50%` makes a box half as tall as it is wide, and why `aspect-ratio: 16 / 9` replaced that trick.

Fluid values

CSS

```css
h1 { font-size: clamp(1.75rem, 4vw + 1rem, 3rem); }
.card { width: min(100% - 2rem, 60ch); }
p { max-width: max(45ch, 90%); }
```

`clamp(min, preferred, max)` clamps the preferred value between the two bounds.

Intrinsic sizing

CSS

```css
min-content   /* narrowest without overflow */
max-content   /* widest without wrapping */
fit-content(20rem) /* min(max-content, 20rem) */
```

6.3 Examples

Example 1: `rem` versus `em`.

CSS

```css
html { font-size: 100%; }        /* usually 16px */
.parent { font-size: 2rem; }     /* 32px */
.parent .child { font-size: 0.5em; }   /* 16px: half of 32px */
```

Expected result: the child ends up at 16px, half the parent's size, while `rem` would always mean half the root size.

Example 2: a responsive card that never exceeds a readable width.

CSS

```css
.card {
  width: min(100% - 2rem, 60ch);
  margin-inline: auto;
  aspect-ratio: 4 / 3;
  object-fit: cover;
}
```

Expected result: the card is 2rem narrower than its container, never wider than 60 characters, centred, with a 4:3 ratio.

Example 3: fluid type.

CSS

```css
h1 {
  font-size: clamp(1.5rem, 1rem + 2vw, 2.5rem);
  line-height: 1.15;
}
```

Expected result: the heading grows with the viewport but never below 1.5rem or above 2.5rem.

Example 4: line length.

CSS

```css
p {
  max-width: 70ch;
  text-wrap: pretty;
}
```

Expected result: paragraphs stay near 66-72 characters per line, which is the comfortable reading range, and widows are softened.

6.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`px` everywhere	ignores user font settings	`rem` for type and spacing
`em` for component spacing	compounds unexpectedly	`rem`
`100vh` on mobile	wrong when toolbars hide	`100dvh` or `svh`
Fixed `height` on text boxes	clipping	`min-height`
`width: 100%` plus padding	overflow	`box-sizing: border-box`
Deeply nested percentage widths	rounding drift	grid or `flex: 1`

- Use `svh`/`dvh`/`lvh` instead of `vh` for full-height sections.
- Keep a spacing scale (4, 8, 12, 16, 24, 32, 48, 64) as custom properties.

6.5 Practice exercises

1. What is `0.5rem` when the root font size is `20px`?
2. Size an element to "60 characters wide, but never wider than its container".
3. Write a fluid font size between `1rem` and `2rem`.
4. Why is `padding-top: 50%` a square-making trick?
5. Give a box a 16:9 aspect ratio.

Solutions

1. `10px`.
2. `width: min(60ch, 100%);`
3. `font-size: clamp(1rem, 2vw + 0.5rem, 2rem);`
4. Percentage padding resolves against the containing block's width, so a `padding-top` of 50% equals half the width, producing a square (or a ratio).
5. `aspect-ratio: 16 / 9;`

6.6 Mini test

1. Multiple choice: which unit is relative to the root font size? - a) `em` - b) `rem` - c) `ch` - d) `vw`
2. True or false: `padding-top: 50%` resolves against the parent height.

3. Multiple choice: which unit measures the width of the digit zero? - a) `ex` - b) `ch` - c) `rem` - d) `cap`
4. Short code question: a heading between 1.25rem and 2rem, scaling with the viewport.
5. Multiple choice: which unit is best for full-height mobile sections? - a) `100vh` - b) `100dvh` - c) `100%` - d) `100em`
6. True or false: `min-content` is the narrowest size without overflow.
7. Short answer: what does `aspect-ratio: 1` do?

Answer key and explanations

1. b - `rem`.
2. False - it resolves against the containing block's width.
3. b - `ch`.
4. `font-size: clamp(1.25rem, 2vw, 2rem);`
5. b - `dvh` follows the dynamic toolbars.
6. True - it is the size where nothing overflows horizontally.
7. Makes the box square: one unit of width per unit of height.

Chapter 7 - Layout, positioning and stacking

7.1 Summary

- Normal flow stacks blocks inline, and inlines flow horizontally.
- `position: static | relative | absolute | fixed | sticky`.
- Absolutely positioned elements need a positioned ancestor.
- `static` is the default; `relative` offsets without leaving the flow.
- `z-index` only works on positioned elements (or flex/grid items).
- Stacking contexts form a tree: `z-index: 9999` cannot escape its context.
- `inset` is the shorthand for `top`, `right`, `bottom`, `left`.
- `float` survives for text wrapping around images.
- Choose a layout method: flow, flex, grid, or positioning.

7.2 Concepts

Positioning

Value	Reference	Space kept?
`static`	normal flow	yes (default)

Value	Reference	Space kept?
`relative`	itself	yes
`absolute`	nearest positioned ancestor	no
`fixed`	the viewport	no
`sticky`	scroll container	yes, until threshold

CSS

```css
.wrap { position: relative; }
.wrap .badge {
  position: absolute;
  top: 0; right: 0;
  inset-inline-end: 0;   /* logical: right in ltr, left in rtl */
}
.to-top {
  position: sticky;
  bottom: 1rem;
}
```

Stacking contexts

A stacking context is created by (among others): `position` + `z-index`, `opacity < 1`, `transform`, `filter`, `will-change`, `isolation: isolate`, `mix-blend-mode`, `contain: paint`, and elements with `position: fixed` or `sticky`.

Inside a context, `z-index` is only compared with its siblings. A child can never paint above a later sibling of its parent, no matter how large its `z-index` is.

CSS

```css
.panel { isolation: isolate; }     /* a predictable wrapper */
.panel > .glow { z-index: 1; }
.panel > .content { z-index: 2; }
```

Float

CSS

```css
figure img { float: left; margin: 0 1rem 0.5rem 0; }
.clearfix::after { content: ""; display: table; clear: both; }
```

Floats are for wrapping text around media. For layout, use flex or grid.

7.3 Examples

Example 1: a badge positioned in a card corner.

CSS

```css
.card { position: relative; }
.card .badge {
  position: absolute;
  top: 0.5rem;
  inset-inline-end: 0.5rem;
}
```

Expected result: the badge sits in the top-right corner (top-left in RTL) and does not push the card content.

Example 2: sticky table header.

CSS

```css
thead th {
  position: sticky;
  top: 0;
  background: #f6f8fa;
  z-index: 1;
}
```

Expected result: the header row stays visible while the table body scrolls.

Example 3: why `z-index: 9999` sometimes does nothing.

CSS

```css
.a { position: relative; z-index: 1; }
.a .child { position: relative; z-index: 9999; }
.b { position: relative; z-index: 2; }
```

HTML

```html
<div class="a"><div class="child">child</div></div>
<div class="b">b</div>
```

Expected result: `.b` paints above the "child" element, because `.child` lives inside the `.a` stacking context, which is below `.b`.

Example 4: a modal overlay.

CSS

```css
.backdrop {
  position: fixed;
  inset: 0;
  background: rgb(0 0 0 / 0.5);
}
dialog::backdrop {
  background: rgb(0 0 0 / 0.5);
}
```

Expected result: the overlay covers the viewport, and the top layer keeps the dialog above it.

7.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`position: absolute` with no positioned ancestor	relative to the page	add `position: relative` to the parent
`z-index` wars	fragile	isolate contexts, keep values small
`position: absolute` for centring	awkward	flex or grid
`top: 50%` without a transform	off by half the element	`translate(-50%, -50%)`
`float` for layout	breaks on small screens	flex/grid
`position: fixed` overlays that trap focus	accessibility trap	use `<dialog>`

- Use `inset-inline` properties so RTL works for free.
- Prefer `<dialog>` for modals: it handles the top layer, focus and `Esc`.

7.5 Practice exercises

1. Position a badge at the bottom-left of a positioned card.
2. Why does `z-index: 9999` fail inside a stacking context?
3. Make a table header sticky.
4. Write the shorthand that sets all four offsets to zero.
5. When is `float` still the right tool?

Solutions

1. `css .card { position: relative; } .badge { position: absolute; bottom: 0.5rem; inset-inline-start: 0.5rem; }`
2. Because `z-index` is only compared inside the nearest stacking context. If an ancestor creates a context with a lower `z-index` than its sibling, all of that subtree paints below the sibling regardless of the child's value.
3. `thead th { position: sticky; top: 0; }`
4. `inset: 0;`
5. When text must wrap around a floated image, which is its intended purpose.

7.6 Mini test

1. Multiple choice: which value keeps the element in the flow? - a) `absolute` - b) `fixed` - c) `relative` - d) `sticky`
2. True or false: `z-index` works on `position: static` elements.
3. Multiple choice: an absolutely positioned element is positioned relative to... - a) the viewport - b) the nearest positioned ancestor - c) `<html>` - d) its parent, always

4. Short code question: stretch a fixed overlay over the whole viewport.
5. Multiple choice: which property creates a stacking context? - a) `overflow` - b) `isolation: isolate` - c) `position: static` - d) `float: none`
6. True or false: `position: sticky` needs a scroll container.
7. Short answer: which logical property pair replaces `left`/`right`?

Answer key and explanations

1. c - `relative` offsets from the normal flow position and keeps the space.
2. False - `z-index` needs a positioned element (or a flex/grid item).
3. b - the nearest ancestor whose `position` is not `static`.
4. `position: fixed; inset: 0;`
5. b - `isolation: isolate` creates a context.
6. True - it sticks inside its scrolling ancestor.
7. `inset-inline-start` and `inset-inline-end`.

Chapter 8 - Flexbox

8.1 Summary

- Flexbox lays out items in one dimension: a row or a column.
- `display: flex` makes the children flex items.
- `flex-direction` sets the main axis; `justify-content` aligns on it.
- `align-items` and `align-self` align on the cross axis.
- `flex-wrap` allows multiple lines; `gap` spaces items without margins.
- `flex: <grow> <shrink> <basis>` is the child-side shorthand.
- `min-width: 0` / `min-height: 0` fixes items that refuse to shrink.
- Flexbox is ideal for toolbars, cards, media objects, centring.
- Reach for Grid when you have rows and columns.

8.2 Concepts

Container properties

Property	Values
`flex-direction`	`row`, `row-reverse`, `column`, `column-reverse`
`flex-wrap`	`nowrap`, `wrap`, `wrap-reverse`
`flex-flow`	shorthand for direction + wrap

Property	Values
`justify-content`	`flex-start`, `center`, `space-between`, `space-around`, `space-evenly`
`align-items`	`stretch`, `flex-start`, `center`, `baseline`, `flex-end`
`align-content`	multi-line spacing on the cross axis
`gap` / `row-gap` / `column-gap`	space between items

Item properties

Property	Effect
`flex-grow`	share of free space
`flex-shrink`	willingness to shrink
`flex-basis`	the starting size before growth
`flex: 1`	`flex-grow: 1; flex-shrink: 1; flex-basis: 0%`
`align-self`	overrides `align-items` for one item
`order`	visual order (keep it in sync with the DOM)
`min-width: 0`	allows shrinking below content size

The two axes

output

```
main axis  ->  flex-direction: row
cross axis   (vertical)

main axis
  |
  v               flex-direction: column
cross axis      (horizontal)
```

`justify-content` always works on the main axis; `align-*` on the cross axis.

The most useful one-liner

CSS

```css
.center {
  display: flex;
  align-items: center;
  justify-content: center;
}
```

(or `display: grid; place-items: center;`)

8.3 Examples

Example 1: a responsive toolbar.

CSS

```css
.toolbar {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 0.5rem 1rem;
  justify-content: space-between;
}
.toolbar .spacer { flex: 1; }
```

HTML

```html
<div class="toolbar">
  <h2>Invoices</h2>
  <span class="spacer"></span>
  <button>Filter</button>
  <button>Export</button>
</div>
```

Expected result: the title on the left, the buttons on the right, wrapping onto a new line on narrow screens, with consistent gaps.

Example 2: equal-width cards that never overflow.

CSS

```css
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
}
.card {
  flex: 1 1 260px;   /* grow, shrink, basis 260px */
  min-width: 0;
}
```

Expected result: cards that are at least 260px wide, wrap when they do not fit, and share leftover space equally.

Example 3: a media object.

CSS

```css
.media {
  display: flex;
  align-items: flex-start;
  gap: 1rem;
}
.media img { flex: 0 0 auto; }
.media .body { flex: 1 1 auto; min-width: 0; }
```

Expected result: an image at a fixed size next to text that fills the rest and wraps inside its column.

Example 4: centring a modal.

CSS

```css
.modal-layer {
  display: grid;
  place-items: center;
  min-height: 100dvh;
}
```

Expected result: the dialog is centred both ways, including on tall phone screens where `100vh` overshoots.

8.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Using flex for a two-dimensional page layout	fights the model	use Grid
`flex: 1` with long words and no `min-width: 0`	horizontal overflow	add `min-width: 0` and `overflow-wrap`
Margins instead of `gap`	asymmetric edges	`gap`
`align-items: stretch` fighting fixed heights	squashed images	`align-items: flex-start`
`order` to "fix" the DOM	confusing for assistive tech	fix the DOM order
Forgetting flex items need `min-height: 0` in columns	clipped content	add it

- Flex items shrink by default; give `flex-shrink: 0` to anything that must keep its size (icons, avatars).

8.5 Practice exercises

1. Centre a div inside a flex container, both axes.
2. Make three children share space equally.
3. Give cards a minimum width of 200px that wrap.
4. Why does long text overflow a `flex: 1` item?
5. Space five items with a gap of 8px.

Solutions

1. `display: flex; align-items: center; justify-content: center;`
2. `.child { flex: 1; }`
3. `.cards { display: flex; flex-wrap: wrap; gap: 1rem; } .card { flex: 1 1 200px; }`
4. Because the default `min-width: auto` keeps flex items at their content size; set `min-width: 0` (plus `overflow-wrap: anywhere`) to let them shrink.
5. `.items { display: flex; gap: 8px; }`

8.6 Mini test

1. Multiple choice: which property aligns items on the main axis? - a) `align-items` - b) `justify-content` - c) `align-content` - d) `place-items`
2. True or false: flexbox is a two-dimensional layout system.
3. Multiple choice: what does `flex: 1` expand to? - a) `flex-grow: 1` only - b) `1 1 0%` - c) `1 1 auto` - d) `1 0 0`
4. Short code question: two equal columns of a flex row.
5. Multiple choice: which property prevents an item from shrinking? - a) `flex-grow: 0` - b) `flex-shrink: 0` - c) `flex-basis: auto` - d) `flex: initial`
6. True or false: `gap` works on flex containers.
7. Short answer: which unit of `min-width` fixes text overflow in flex?

Answer key and explanations

1. b - `justify-content` is the main axis.
2. False - that is Grid; flex is one-dimensional.
3. b - `1 1 0%`.
4. `.col { flex: 1; }` (or `flex: 1 1 0`)
5. b - `flex-shrink: 0`.
6. True - `gap`, `row-gap` and `column-gap` all apply.
7. `min-width: 0` (and `min-height: 0` for columns).

Chapter 9 - Grid

9.1 Summary

- Grid lays out in two dimensions: rows and columns at once.
- `display: grid` plus `grid-template-columns`/`grid-template-rows`.
- `fr` is a fraction of leftover space; `minmax()` bounds a track.
- `repeat()` and `auto-fit`/`auto-fill` build responsive grids without media queries.
- Children place themselves with `grid-column`/`grid-row` and line numbers.
- Named lines and named areas make layouts readable.
- `subgrid` shares a parent's tracks with a child.
- Grid is ideal for page layouts, dashboards, and image galleries.
- Grid items stretch by default; `align-self`/`justify-self` override it.

9.2 Concepts

Defining tracks

CSS

```css
grid-template-columns: 1fr 2fr;              /* two columns */
grid-template-columns: repeat(3, 1fr);       /* three equal */
grid-template-columns: 200px 1fr 200px;      /* fixed sides, fluid middle */
grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
grid-template-rows: auto 1fr auto;
gap: 1rem;   /* also row-gap and column-gap */
```

`1fr` means "one part of the free space", not "25%".

Placing items

CSS

```css
.item {
  grid-column: 1 / 3;    /* start line 1, end line 3 (two tracks) */
  grid-row: span 2;
  grid-column: 2 / -1;   /* last line is -1 */
}
```

Named areas

CSS

```css
.page {
  display: grid;
  grid-template-areas:
    "head head"
    "nav  main"
    "foot foot";
  grid-template-columns: 200px 1fr;
  grid-template-rows: auto 1fr auto;
  min-height: 100dvh;
}
.page > header { grid-area: head; }
.page > nav    { grid-area: nav; }
.page > main   { grid-area: main; }
.page > footer { grid-area: foot; }
```

Alignment

CSS

```css
justify-items: start;    /* inline axis (default stretch) */
align-items: center;     /* block axis */
place-items: center;     /* both */
justify-content / align-content;  /* the whole grid in its container */
```

9.3 Examples

Example 1: the "holy grail" layout in twelve lines.

CSS

```css
.layout {
  display: grid;
  grid-template:
    "header header" auto
    "nav    main"   1fr
    "footer footer" auto
    / 12rem 1fr;
  min-height: 100dvh;
}
.layout > header { grid-area: header; }
.layout > nav { grid-area: nav; }
.layout > main { grid-area: main; }
.layout > footer { grid-area: footer; }
```

Expected result: a full-height page with a header, a 12rem sidebar, a fluid main area and a footer that stays at the bottom.

Example 2: a responsive gallery with no media queries.

CSS

```css
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(180px, 1fr));
  gap: 12px;
}
```

Expected result: as many 180px columns as fit, sharing the leftover space, so the gallery adapts to every screen width automatically.

Example 3: a card with a sticky footer.

CSS

```css
.card {
  display: grid;
  grid-template-rows: auto 1fr auto;  /* media, body, actions */
}
```

Expected result: the card body grows so the action buttons align at the bottom of cards of different heights.

Example 4: spanning an item.

CSS

```css
.featured { grid-column: span 2; grid-row: span 2; }
```

Expected result: the featured tile occupies two columns and two rows.

9.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`grid-template-columns: 25% 25% 25%` with gaps	overflow, no leftover space	`1fr`
Nesting grids for a simple row	complexity	flex inside a grid cell

Mistake	Why it hurts	Best practice
`auto-fill` with huge `minmax`	empty tracks	`auto-fit` or a smaller minimum
Fixed row heights	clipped content	`auto` or `minmax(auto, 1fr)`
Line-number placement everywhere	fragile	named areas
Forgetting `min-height: 0` on `1fr` children	content overflow	add it

- Use `minmax(min(240px, 100%), 1fr)` for cards that must never exceed the viewport on tiny screens.
- `subgrid` is the answer when nested children must line up with the parent.

9.5 Practice exercises

1. Write a three-column equal grid with a 16px gap.
2. Make a grid that adds columns automatically with a 200px minimum.
3. Name four areas for a header, sidebar, main and footer.
4. Which value specifies a flexible fraction?
5. Make a grid child span two columns.

Solutions

1. `display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px;`
2. `grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));`
3. `css grid-template-areas: "header header" "sidebar main" "footer footer";`
4. `fr`.
5. `grid-column: span 2;`

9.6 Mini test

1. Multiple choice: which unit is a fraction of leftover space? - a) `%` - b) `fr` - c) `em` - d) `vw`
2. True or false: Grid can align rows and columns at the same time.
3. Multiple choice: which keyword adds as many tracks as fit? - a) `repeat()` - b) `auto-fit` - c) `subgrid` - d) `auto-fill` and `auto-fit`
4. Short code question: a grid whose items are at least 160px wide and grow.
5. Multiple choice: which property aligns an item inside its grid area on the block axis? - a) `justify-self` - b) `align-self` - c) `place-content` - d) `order`
6. True or false: grid items stretch to fill their area by default.
7. Short answer: what does `subgrid` do?

Answer key and explanations

1. b - `fr`.

2. True - that is what makes it two-dimensional.
3. d - both `auto-fill` and `auto-fit`.
4. `grid-template-columns: repeat(auto-fill, minmax(160px, 1fr));`
5. b - `align-self` (block axis); `justify-self` is inline.
6. True - unless `align-items`/`align-self` change it.
7. It makes the child inherit the parent's track sizing, so grandchildren can align to the same rows and columns.

Chapter 10 - Logical properties, responsive design, container queries

10.1 Summary

- Logical properties map to physical sides based on writing mode and direction.
- `margin-inline-start` is `left` in LTR and `right` in RTL.
- `text-align: start` and `inset-inline` replace `left`/`right`.
- Media queries target viewport size, hover, prefers-reduced-motion, print.
- Mobile-first: base styles for small screens, `min-width` for larger.
- Container queries style a component by its container's size, not the viewport.
- `@container` requires `container-type` on an ancestor.
- `:has()` plus container queries replace many JavaScript patterns.
- Responsive design is about content, not device lists.

10.2 Concepts

Logical properties map

Physical	Logical
`margin-left` / `margin-right`	`margin-inline-start` / `margin-inline-end`
`padding-top` / `padding-bottom`	`padding-block-start` / `padding-block-end`
`border-left`	`border-inline-start`
`top` / `bottom` (inset)	`inset-block-start` / `inset-block-end`
`left` / `right` (inset)	`inset-inline-start` / `inset-inline-end`
`width` / `height`	`inline-size` / `block-size`
`text-align: left`	`text-align: start`
`float: left`	`float: inline-start`

Shorthands: `margin-inline`, `margin-block`, `padding-inline`, `inset-inline`, `border-inline`.

Writing mode

CSS

```css
.vertical { writing-mode: vertical-rl; }
.rtl { direction: rtl; }
```

With `direction: rtl`, `margin-inline-start` automatically flips.

Media queries

CSS

```css
@media (min-width: 40em) { }            /* width */
@media (max-width: 30em) { }            /* max width */
@media (hover: hover) { }               /* input capability */
@media (prefers-reduced-motion: reduce) { }
@media (prefers-color-scheme: dark) { }
@media print { }
@media (orientation: landscape) { }
```

Container queries

CSS

```css
.sidebar { container-type: inline-size; container-name: sidebar; }

@container sidebar (min-width: 30rem) {
  .card { grid-template-columns: 12rem 1fr; }
}

/* size units available in container queries */
.title { font-size: clamp(1.5rem, 5cqi, 2.5rem); }
```

`cqi`, `cqb`, `cqw`, `cqh` and `cqmin`/`cqmax` are container-relative units.

10.3 Examples

Example 1: a layout that works in RTL for free.

CSS

```css
.callout {
  border-inline-start: 4px solid #0969da;
  padding-inline-start: 1rem;
  text-align: start;
}
```

Expected result: in an LTR page the bar is on the left; when `<html dir="rtl">`, it moves to the right with no extra CSS.

Example 2: mobile-first responsive columns.

CSS

```css
.cards { display: grid; gap: 1rem; }

@media (min-width: 40em) {
  .cards { grid-template-columns: repeat(2, 1fr); }
}

@media (min-width: 60em) {
  .cards { grid-template-columns: repeat(3, 1fr); }
}
```

Expected result: one column on phones, two on tablets, three on desktops.

Example 3: a component that responds to its container.

CSS

```css
.card { container-type: inline-size; }

@container (min-width: 30rem) {
  .card .body {
    display: grid;
    grid-template-columns: 10rem 1fr;
    gap: 1rem;
  }
}
```

Expected result: the same card switches to a horizontal layout whether it is in a wide sidebar or a narrow column, with no knowledge of the viewport.

Example 4: respecting reduced motion.

CSS

```css
.spinner { animation: spin 1s linear infinite; }

@media (prefers-reduced-motion: reduce) {
  .spinner { animation-duration: 4s; }
  * { scroll-behavior: auto !important; }
}
```

Expected result: users who ask for less motion get a slower spin instead of none.

10.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`left`/`right` for spacing	breaks in RTL	logical properties
Desktop-first media queries	more overrides	mobile-first with `min-width`
Device breakpoint lists	brittle	content-driven breakpoints
`container-type` on scrollable boxes	sizing bugs	only where needed
Ignoring `prefers-reduced-motion`	accessibility failure	honour it
Hiding content on mobile instead of reflowing	broken UX	reflow

- Use `em`-based breakpoints so zoom does not break the layout.
- Add `container-type: inline-size` to reusable components first.

10.5 Practice exercises

1. Write a left margin that works in both LTR and RTL.
2. Write a min-width media query at 48em.
3. Make a card's container respond to a 25rem minimum width.
4. Which media feature detects "no motion, please"?
5. Why prefer mobile-first?

Solutions

1. `margin-inline-start: 1rem;`
2. `@media (min-width: 48em) { }`
3. `css .card { container-type: inline-size; } @container (min-width: 25rem) { .card { ... } }`
4. `prefers-reduced-motion: reduce`.
5. Because the base styles then serve the smallest, slowest devices, and each breakpoint only adds. That produces less CSS and fewer overrides than starting from desktop and undoing.

10.6 Mini test

1. Multiple choice: which property is logical? - a) `margin-left` - b) `margin-inline-start` - c) `left` - d) `width`
2. True or false: `inset-inline-end` is `left` in RTL.
3. Multiple choice: what does `container-type: inline-size` allow? - a) media queries - b) `@container` queries on the element's width - c) `position: container` - d) scrolling
4. Short code question: a dark-mode override.
5. Multiple choice: which unit is container-relative? - a) `vw` - b) `cqi` - c) `rem` - d) `ch`
6. True or false: `@container` needs a container-type on an ancestor.
7. Short answer: which media feature should disable smooth scrolling?

Answer key and explanations

1. b - `margin-inline-start`.
2. True - logical inline end maps to the right in LTR and the left in RTL.
3. b - it lets `@container` query the element's inline size.
4. `css @media (prefers-color-scheme: dark) { body { background: #101418; } }`
5. b - `cqi`.
6. True - without it there is no container to query.
7. `prefers-reduced-motion`.

Chapter 11 - Custom properties and functions

11.1 Summary

- Custom properties start with `--` and are read with `var()`.
- `--*: value` declares; `var(--name, fallback)` consumes.
- They cascade and inherit like any other property.
- They can be redefined per component, per state, or per breakpoint.
- Unlike Sass variables, they are live: change them at runtime, including from JavaScript.
- `@property` declares type, syntax and initial value, and enables animation.
- `calc()` does maths on mixed units; `clamp()`, `min()`, `max()` bound results.
- `env(safe-area-inset-*)` handles notches and home indicators.
- Custom properties are not animatable unless `@property` gives them a type.
- Design tokens are just custom properties with discipline.

11.2 Concepts

Defining and using

CSS

```css
:root {
  --space-1: 0.25rem;
  --space-2: 0.5rem;
  --radius: 8px;
  --accent: #0969da;
  --shadow: 0 1px 3px rgb(0 0 0 / 0.12);
}

.button {
  padding: var(--space-2) var(--space-2);
  border-radius: var(--radius);
  background: var(--accent);
  box-shadow: var(--shadow);
}
```

Runtime changes

CSS

```css
:root { --accent: #0969da; }
[data-theme="dark"] { --accent: #58a6ff; }
.button { background: var(--accent); }
```

HTML

```html
<html data-theme="dark">
```

One attribute changes the whole theme, with no duplicated rules.

Typed custom properties with `@property`

CSS

```css
@property --angle {
  syntax: "<angle>";
  inherits: false;
  initial-value: 0deg;
}

.spinner {
  --angle: 0deg;
  background: conic-gradient(from var(--angle), #0969da, #8250df);
  animation: rotate-angle 2s linear infinite;
}
@keyframes rotate-angle {
  to { --angle: 360deg; }
}
```

Typed properties can be interpolated, which is how you animate gradients.

Maths functions

CSS

```css
width: calc(100% - 2rem);
font-size: clamp(1rem, 0.5rem + 1vw, 1.5rem);
padding: max(1rem, 3vw);
width: min(90%, 60ch);
inset: env(safe-area-inset-top, 0px);
```

11.3 Examples

Example 1: a design token set.

CSS

```css
:root {
  /* colour */
  --canvas: #ffffff;
  --ink: #1f2328;
  --muted: #57606a;
  --accent: #0969da;
  --border: #d8dee4;
  /* space */
  --space-1: 0.25rem;
  --space-2: 0.5rem;
  --space-3: 1rem;
  --space-4: 1.5rem;
  /* type */
  --font-sans: system-ui, "Segoe UI", sans-serif;
  --font-mono: ui-monospace, "Cascadia Code", monospace;
}

.card {
  background: var(--canvas);
  color: var(--ink);
  border: 1px solid var(--border);
  padding: var(--space-4);
  font-family: var(--font-sans);
}
```

Expected result: a card whose every visual decision comes from a token, so retuning the palette only touches `:root`.

Example 2: a dark theme with the same rules.

CSS

```css
:root { color-scheme: light dark; }

@media (prefers-color-scheme: dark) {
  :root {
    --canvas: #0d1117;
    --ink: #e6edf3;
    --muted: #9198a1;
    --accent: #58a6ff;
    --border: #30363d;
  }
}
```

Expected result: the card automatically follows the operating system theme.

Example 3: theming a component in isolation.

CSS

```css
:root { --btn-bg: var(--accent); }
.btn-danger { --btn-bg: #cf222e; }   /* only this component changes */
.btn { background: var(--btn-bg); }
```

Expected result: a "danger" button recolours without a second `.btn` rule.

Example 4: `calc()` with a fallback.

CSS

```css
.sidebar {
  width: calc(var(--sidebar, 16rem) - var(--space-3));
}
```

If `--sidebar` is unset the declaration still resolves to `16rem - 1rem`.

11.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`var(--x)` with no declaration	invalid at computed-value time	always declare, or give a fallback
Custom properties holding colours only	misses spacing/type	full token set
Interpolating custom properties in animations	no effect	declare with `@property`
Redefining tokens deep in the tree	confusing provenance	redefine at theme boundary
`var()` inside `url()` without quotes	parsing issues	`url("...")`
Unit mismatch in `calc()`	invalid	`+/-` need matching units

- Name tokens by role (`--accent`), not by value (`--blue`).
- Keep `@property` declarations in a single, documented file.

11.5 Practice exercises

1. Declare a `--radius` custom property and use it for two components.
2. Give a custom property a fallback.
3. Animate a custom property. What must you add?
4. Write a width that is `100%` minus the sidebar.
5. Why name tokens by role?

Solutions

1. `:root { --radius: 8px; }` then `border-radius: var(--radius);`
2. `color: var(--ink, #000);`
3. An `@property` declaration with a `syntax` type (e.g. `"<length>"`), so the value can be interpolated.
4. `width: calc(100% - 16rem);`
5. Because the token is a role in the system: renaming the palette (from blue to teal) should not require renaming every usage.

11.6 Mini test

1. Multiple choice: how do you read a custom property? - a) `$name` - b) `var(--name)` - c) `#name` - d) `@name`
2. True or false: custom properties can be changed from JavaScript at runtime.
3. Multiple choice: which at-rule types a custom property? - a) `@media` - b) `@property` - c) `@layer` - d) `@keyframes`
4. Short code question: a width of `100%` minus `2rem`.
5. Multiple choice: which function clamps a value between two bounds? - a) `min()` - b) `clamp()` - c) `calc()` - d) `max()`
6. True or false: `var()` may be used inside any other function.
7. Short answer: what happens when a custom property is used but never declared?

Answer key and explanations

1. b - `var(--name)`.
2. True - `el.style.setProperty("--accent", "red")` works immediately.
3. b - `@property`.
4. `width: calc(100% - 2rem);`
5. b - `clamp(min, preferred, max)`.
6. True - with the usual rule that the final value must be valid for the property it lands in.
7. The declaration becomes "invalid at computed-value time": the property takes its inherited or initial value.

Chapter 12 - Typography, lists, counters

12.1 Summary

- Set a font stack, a comfortable `line-height`, and a `max-width` per block.
- `line-height: 1.5` is the classic body value; headings want less.
- Use `text-wrap: balance` on headings, `pretty` on paragraphs.
- `font-size-adjust`, `letter-spacing` and `word-spacing` refine rhythm.
- `::marker` styles list bullets, and `counter-reset`/`counter-increment` build custom numbering.
- `list-style-type` accepts strings: `list-style-type: "-- "`.
- `text-overflow`, `overflow-wrap: anywhere` and `hyphens: auto` handle breaking.
- `::first-line` and `::first-letter` add magazine touches.
- Vertical rhythm comes from consistent spacing tokens.
- Tabular numbers (`font-variant-numeric: tabular-nums`) align figures.

12.2 Concepts

A readable base

CSS

```css
body {
  font-family: var(--font-sans);
  font-size: 1rem;        /* never smaller than the user default */
  line-height: 1.5;
  color: var(--ink);
}

h1, h2, h3 {
  line-height: 1.15;
  text-wrap: balance;
  letter-spacing: -0.01em;
}

p {
  max-width: 70ch;
  text-wrap: pretty;
}
```

Font stack

CSS

```css
--font-sans: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
--font-mono: ui-monospace, "Cascadia Code", "Courier New", monospace;
```

A font stack is a list of fallbacks; the browser uses the first available.

Lists and markers

CSS

```css
::marker {
  color: var(--accent);
  content: "\2192  ";      /* a custom marker */
}
.list-simple { list-style: none; padding: 0; }
```

Counters

CSS

```css
.reset-count { counter-reset: section; }
.item { counter-increment: section; }
.item::before {
  content: counter(section) ". ";
  font-weight: bold;
}
```

12.3 Examples

Example 1: a typographic base.

CSS

```css
:root {
  --font-sans: system-ui, sans-serif;
  --font-mono: ui-monospace, monospace;
  --step-0: 1rem;
  --step-1: 1.25rem;
  --step-2: 1.5rem;
}

body {
  font-family: var(--font-sans);
  font-size: var(--step-0);
  line-height: 1.5;
}

h1 { font-size: var(--step-2); line-height: 1.15; text-wrap: balance; }
h2 { font-size: var(--step-1); line-height: 1.2; }
```

Expected result: a clear size hierarchy with tighter heading leading, and headings that break evenly across lines.

Example 2: a code block.

CSS

```css
pre, code {
  font-family: var(--font-mono);
  font-size: 0.9em;
  tab-size: 2;
}
pre {
  background: #f6f8fa;
  padding: 1rem;
  border-radius: 8px;
  overflow-x: auto;
}
```

Expected result: monospaced code with two-space tabs, a light background and a horizontal scrollbar for long lines.

Example 3: custom counters for a recipe.

CSS

```css
.ingredients { counter-reset: item; list-style: none; }
.ingredients li { counter-increment: item; }
.ingredients li::before {
  content: counter(item) "\2002";   /* an en space after the number */
  color: var(--accent);
  font-weight: 700;
}
```

Expected result: a numbered list with accent-coloured numbers, generated by CSS rather than typed in the HTML.

Example 4: `::first-letter`.

CSS

```css
article > p:first-of-type::first-letter {
  font-size: 3rem;
  line-height: 0.8;
  float: inline-start;
  padding-inline-end: 0.25rem;
}
```

Expected result: a drop cap on the opening paragraph, aligned correctly in both LTR and RTL.

12.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Body text below 16px	unreadable, ignores user settings	`1rem` minimum
`line-height: 1` on body text	crowded lines	`1.5`
Lines 120+ characters	loses the reader	`max-width: 70ch`
`float` drop caps	awkward reflow	`float: inline-start`
`text-transform: uppercase` in HTML	changes the text itself	CSS only
`word-break: break-all`	breaks words mid-token	`overflow-wrap: anywhere`
Justified text without hyphens	rivers of white space	`hyphens: auto`

- Use `font-variant-numeric: tabular-nums` in tables and timers.
- Do not disable zoom; respect the user's font size.

12.5 Practice exercises

1. Set a 1.5 line height and a 70-character maximum width on all paragraphs.
2. Make a custom marker for list items.
3. Number a list of sections with CSS counters.
4. Style code blocks monospaced with a scrollbar when needed.
5. Which two properties prevent long URLs from breaking a layout?

Solutions

1. `p { line-height: 1.5; max-width: 70ch; }`
2. `li::marker { color: var(--accent); content: "\2192 "; }`
3. `css .sections { counter-reset: s; } .sections li { counter-increment: s; } .sections li::before { content: counter(s) ". "; }`
4. `css pre { font-family: var(--font-mono); overflow-x: auto; }`
5. `overflow-wrap: anywhere` (or `break-word`) and `min-width: 0` on flex items.

12.6 Mini test

1. Multiple choice: which property sets the space between baselines relative to font size? - a) `line-height` - b) `letter-spacing` - c) `leading` - d) `baseline`
2. True or false: `text-wrap: balance` improves heading line breaks.
3. Multiple choice: which pseudo-element styles the list bullet? - a) `::before` - b) `::marker` - c) `::bullet` - d) `::first-letter`
4. Short code question: limit paragraphs to about 70 characters per line.
5. Multiple choice: which property creates a custom counter? - a) `counter-increment` - b) `counter-reset` - c) `counter()` - d) both a and b
6. True or false: `word-break: break-all` is the safest way to break long URLs.
7. Short answer: why prefer `float: inline-start` for drop caps?

Answer key and explanations

1. a - `line-height` (unitless, so it scales with the font).
2. True - it balances line lengths in headings in modern browsers.
3. b - `::marker`.
4. `p { max-width: 70ch; }`
5. d - `counter-reset` starts it, `counter-increment` advances it.
6. False - `overflow-wrap: anywhere` is targeted at long unbreakable strings; `break-all` breaks ordinary words too.
7. Because it follows the reading direction, so the drop cap is on the correct side in RTL without a media query or a duplicated rule.

Chapter 13 - Borders, shadows, focus

13.1 Summary

- `border` is the shorthand for width, style and colour.
- `border-radius` accepts two values per corner for elliptical corners.
- `box-shadow` takes offsets, blur, spread, colour and `inset`.
- `outline` is for focus; `border` is for decoration.
- `:focus-visible` replaces `:focus` for keyboard-only highlighting.
- `box-shadow` respects `border-radius`; `filter: drop-shadow()` follows real edges.
- Layered shadows look better than one heavy shadow.
- Never remove the focus indicator without a visible replacement.
- `outline-offset` adds breathing room between the box and its ring.

13.2 Concepts

Border shorthand

CSS

```css
border: 2px solid #d8dee4;             /* width style colour */
border-block-start: 3px solid var(--accent);
border-inline-end: 1px solid var(--border);
border-radius: 12px;                   /* 50% makes a circle on a square */
border-radius: 1rem 2rem 3rem 4rem;    /* TL TR BR BL */
border-start-start-radius: 12px;       /* logical corners */
```

Shadows

CSS

```css
box-shadow: 0 1px 2px rgb(0 0 0 / 0.06), 0 4px 12px rgb(0 0 0 / 0.08);
box-shadow: inset 0 1px 0 rgb(255 255 255 / 0.5);
filter: drop-shadow(0 2px 4px rgb(0 0 0 / 0.25));
```

Order: `offset-x offset-y blur spread color`, `inset` optional.

Focus

CSS

```css
:focus-visible {
  outline: 2px solid var(--accent);
  outline-offset: 2px;
  border-radius: inherit;
}
```

`outline` does not affect layout, so it is the safe choice for focus rings; `:focus-visible` avoids a ring on mouse clicks.

13.3 Examples

Example 1: a layered card shadow.

CSS

```css
.card {
  background: var(--canvas);
  border: 1px solid var(--border);
  border-radius: 12px;
  box-shadow:
    0 1px 2px rgb(0 0 0 / 0.04),
    0 8px 24px rgb(0 0 0 / 0.06);
}
.card:hover {
  box-shadow:
    0 2px 4px rgb(0 0 0 / 0.05),
    0 12px 32px rgb(0 0 0 / 0.10);
}
```

Expected result: a card with a subtle shadow that deepens on hover, both layers following the rounded corners.

Example 2: an accessible focus ring.

CSS

```css
.button {
  border: 1px solid var(--border);
  background: var(--canvas);
}
.button:focus-visible {
  outline: 2px solid var(--accent);
  outline-offset: 2px;
}
```

Expected result: clicking does nothing visible; pressing `Tab` draws a blue ring two pixels outside the button.

Example 3: a pill and a circle.

CSS

```css
.pill { border-radius: 999px; }
.avatar { width: 40px; height: 40px; border-radius: 50%; }
```

Expected result: fully rounded ends and a circular avatar.

Example 4: an inset "pressed" effect.

CSS

```css
.pressed {
  box-shadow: inset 0 2px 4px rgb(0 0 0 / 0.15);
  background: #eaeef2;
}
```

Expected result: the element appears recessed instead of raised.

13.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`outline: none`	keyboard users are lost	style `:focus-visible`
One huge blur shadow	muddy, unrealistic	two layers, small blur
`box-shadow` on transparent images	a visible box	`filter: drop-shadow()`
Border colour as the only state signal	inaccessible	icon or text too
`border-radius: 50%` on non-square boxes	an ellipse	pair with equal width/height
`box-shadow` everywhere	slow paints on mobile	use sparingly

- Prefer `outline` for focus so the ring never shifts layout.
- Keep shadow values in tokens (`--shadow-1`, `--shadow-2`).

13.5 Practice exercises

1. Write a 1px solid border in the accent colour with 8px corners.
2. Style `:focus-visible` with a 2px offset ring.
3. Create a circular 48px avatar with a border.

4. Why use `filter: drop-shadow()` for icon images?
5. Write a two-layer shadow.

Solutions

1. `border: 1px solid var(--accent); border-radius: 8px;`
2. `css :focus-visible { outline: 2px solid var(--accent); outline-offset: 2px; }`
3. `css .avatar { width: 48px; height: 48px; border-radius: 50%; border: 2px solid var(--border); object-fit: cover; }`
4. Because `box-shadow` shadows the element's rectangle, while `drop-shadow()` follows the alpha channel, so a transparent PNG gets a silhouette shadow.
5. `box-shadow: 0 1px 2px rgb(0 0 0 / 0.06), 0 4px 12px rgb(0 0 0 / 0.08);`

13.6 Mini test

1. Multiple choice: which property should style keyboard focus? - a) `border` - b) `outline` - c) `box-shadow` - d) `background`
2. True or false: `:focus-visible` shows the ring for mouse clicks too.
3. Multiple choice: which `box-shadow` value draws inside the box? - a) `inset` - b) `outset` - c) `inner` - d) `spread`
4. Short code question: a 999px radius pill.
5. Multiple choice: which shadow follows the real shape of an image? - a) `box-shadow` - b) `filter: drop-shadow()` - c) `text-shadow` - d) `border-shadow`
6. True or false: `outline-offset` moves the ring away from the element.
7. Short answer: why is a single large-blur shadow worse than two layers?

Answer key and explanations

1. b - `outline` (via `:focus-visible`).
2. False - that is the point: it generally appears for keyboard focus.
3. a - `inset`.
4. `border-radius: 999px;`
5. b - `filter: drop-shadow()`.
6. True - it adds space between the outline and the border edge.
7. The eye expects a tight contact shadow plus a wider ambient shadow; one spread blur reads as a dirty halo.

Chapter 14 - Overflow, scrolling, cursors

14.1 Summary

- `overflow` controls what happens when content exceeds its box.

- `overflow: hidden` clips; `scroll` always shows bars; `auto` when needed.
- `overflow-x: hidden` on `html` breaks `position: sticky` inside it.
- `scroll-behavior: smooth` plus `:target` gives anchor animation.
- `scroll-snap` builds carousels and galleries with CSS alone.
- `scrollbar-gutter: stable` avoids layout shift when scrollbars appear.
- `overscroll-behavior` stops scroll chaining to the page.
- `cursor` communicates affordance: `pointer` on interactive, `not-allowed` when disabled.
- `scroll-margin` and `scroll-padding` fix sticky-header anchor offsets.
- `@supports` guards newer features.

14.2 Concepts

Overflow values

Value	Behaviour
`visible`	spills out (default)
`hidden`	clipped, no scrolling
`scroll`	scrollbars always visible
`auto`	scrollbars only when needed
`clip`	clipped, no scroll container

CSS

```css
.scroll-area {
  max-height: 300px;
  overflow-y: auto;
  overscroll-behavior: contain;
}
```

Scroll snapping

CSS

```css
.gallery {
  display: grid;
  grid-auto-flow: column;
  grid-auto-columns: 80%;
  gap: 1rem;
  overflow-x: auto;
  scroll-snap-type: x mandatory;
}
.gallery > * { scroll-snap-align: start; }
```

Smooth scrolling and anchor offsets

CSS

```css
html { scroll-behavior: smooth; }
section { scroll-margin-top: 4rem; }     /* clear the sticky header */
html { scroll-padding-top: 4rem; }
```

Cursors

cursor	Meaning
default	the arrow
pointer	a link or a clickable element
text	selectable text
not-allowed	the action is blocked
grab / grabbing	draggable
zoom-in	opens a viewer
help / wait / progress	status

CSS

```css
@media (prefers-reduced-motion: reduce) {
  html { scroll-behavior: auto; }
}
```

14.3 Examples

Example 1: a scrollable code or log panel.

CSS

```css
.log {
  max-height: 16rem;
  overflow-y: auto;
  padding: 0.75rem 1rem;
  background: var(--canvas);
  border: 1px solid var(--border);
  border-radius: 8px;
  overscroll-behavior: contain;
}
```

Expected result: a fixed-height panel with its own scrollbar; scrolling to the end does not scroll the page behind it.

Example 2: a snap gallery.

CSS

```css
.strip {
  display: flex;
  gap: 1rem;
  overflow-x: auto;
  scroll-snap-type: x mandatory;
  scroll-padding-inline: 1rem;
}
.strip > figure {
  flex: 0 0 min(80%, 20rem);
  scroll-snap-align: start;
}
```

Expected result: a swipeable row of cards that snaps to the start of each item.

Example 3: an anchor that lands below the sticky header.

CSS

```css
.site-header {
  position: sticky;
  top: 0;
  height: 4rem;
  z-index: 10;
}
main > section { scroll-margin-top: 5rem; }
```

Expected result: clicking `#pricing` shows the heading just below the header instead of hiding it underneath.

Example 4: guarding a newer feature.

CSS

```css
.card { display: grid; }
@supports (container-type: inline-size) {
  .card { container-type: inline-size; }
}
```

Expected result: browsers without container queries still get the base grid.

14.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`overflow-x: hidden` on `html`	breaks `position: sticky`	fix the overflow source
`overflow: hidden` to "fix" margins	hides real bugs	find the overflowing child
No `scroll-margin` with a sticky header	headings hidden under it	`scroll-margin-top`
`cursor: pointer` on non-interactive elements	misleading	only on interactive
Always-on `scroll` bars	visual noise	`auto`
Ignoring `prefers-reduced-motion` for smooth scroll	motion sickness	disable it

- `scrollbar-gutter: stable` on `html` prevents a 15px jump on every navigation.
- Use `overflow: clip` when you want clipping without creating a scroll container.

14.5 Practice exercises

1. Create a scrollable panel with a maximum height of 200px.
2. Make a card row snap horizontally.
3. Stop the page from scrolling when a modal body reaches its end.
4. Fix anchors landing under a 4rem sticky header.
5. Which cursor value means "this action is not permitted"?

Solutions

1. `max-height: 200px; overflow-y: auto;`
2. `css .row { display: flex; overflow-x: auto; scroll-snap-type: x mandatory; } .row > * { scroll-snap-align: start; }`
3. `overscroll-behavior: contain;` on the scrollable modal body.
4. `scroll-margin-top: 4rem;` on the targets (or `scroll-padding-top` on `html`).
5. `not-allowed`.

14.6 Mini test

1. Multiple choice: which value clips content without making it scrollable? - a) `hidden` - b) `clip` - c) `scroll` - d) `auto`
2. True or false: `overflow-x: hidden` on `<html>` disables `position: sticky` descendants.
3. Multiple choice: which property makes `scroll-snap` mandatory? - a) `scroll-snap-align` - b) `scroll-snap-type` - c) `scroll-padding` - d) `snap`
4. Short code question: space out anchor targets by 5rem for a sticky header.
5. Multiple choice: which cursor signals a blocked action? - a) `default` - b) `not-allowed` - c) `progress` - d) `no-drop`
6. True or false: `scroll-behavior: smooth` should stay enabled when the user prefers reduced motion.
7. Short answer: what does `scrollbar-gutter: stable` prevent?

Answer key and explanations

1. b - `clip`.
2. True - the scroll container is no longer `html`, so sticky elements lose their reference; use `overflow-x: clip` instead, or fix the cause.
3. b - `scroll-snap-type`.
4. `scroll-margin-top: 5rem;`
5. b - `not-allowed`.

6. False - honour `prefers-reduced-motion: reduce` and set it to `auto`.
7. The layout shift that happens when a scrollbar appears or disappears (the content jumping sideways).

Chapter 15 - Filters, blend modes, clipping and masking

15.1 Summary

- `filter` applies visual effects: blur, brightness, contrast, shadows.
- `backdrop-filter` affects what is behind the element.
- `mix-blend-mode` and `background-blend-mode` mix layers.
- `clip-path` cuts a box into a shape with `circle()`, `ellipse()`, `polygon()` or `path()`.
- `mask-image` (and `mask-mode`) hides parts of an element.
- `clip-path` and `mask` both follow the box, not the alpha of an image.
- Effects are GPU-friendly but still cost: animate `transform` and `opacity` first.
- Always provide a focus-visible state on filtered interactive elements.
- Filters inherit into children, including text.

15.2 Concepts

Filter functions

Function	Example
`blur()`	`blur(4px)`
`brightness()` / `contrast()`	`brightness(1.1) contrast(1.05)`
`grayscale()` / `sepia()`	`sepia(0.5)`
`hue-rotate()`	`hue-rotate(90deg)`
`invert()`	`invert(1)` (dark mode trick)
`saturate()`	`saturate(2)`
`drop-shadow()`	`drop-shadow(0 2px 4px rgb(0 0 0 / 0.25))`

CSS

```css
filter: grayscale(1) blur(0);          /* list of functions, applied in order */
backdrop-filter: blur(8px) saturate(1.2);
```

Blend modes

`mix-blend-mode`	**Effect**
`normal`	no blending
`multiply` / `screen`	darken / lighten
`overlay` / `soft-light`	contrast-aware mix
`difference` / `exclusion`	invert where colours differ
`hue` / `saturation` / `color` / `luminosity`	channel mix

Blending needs a context: `isolation: isolate` keeps it inside a component.

Clip-path shapes

CSS

```css
clip-path: circle(50% at 50% 50%);
clip-path: ellipse(40% 60% at 50% 50%);
clip-path: polygon(0 0, 100% 0, 100% 80%, 50% 100%, 0 80%);
clip-path: path("M0,0 L100,0 L100,100 Z");
```

Masks

CSS

```css
mask-image: url("mask.svg");
mask-mode: alpha;              /* luminance or alpha */
mask-repeat: no-repeat;
mask-size: cover;
```

15.3 Examples

Example 1: a frosted glass panel.

CSS

```css
.glass {
  background: rgb(255 255 255 / 0.6);
  backdrop-filter: blur(10px) saturate(1.4);
  border: 1px solid rgb(255 255 255 / 0.4);
  border-radius: 12px;
}
```

Expected result: the panel blurs and brightens whatever scrolls behind it.

Example 2: a duotone hover on an image.

CSS

```css
.duotone img {
  filter: grayscale(1) contrast(1.1);
  transition: filter 200ms ease;
}
.duotone:hover img,
.duotone:focus-within img {
  filter: none;
}
```

Expected result: images are black and white until the card is hovered or focused.

Example 3: a tag with a clipped corner.

CSS

```css
.tag {
  padding: 0.25rem 0.75rem;
  background: var(--accent);
  color: #fff;
  clip-path: polygon(0 0, 100% 0, calc(100% - 8px) 100%, 0 100%);
}
```

Expected result: a banner whose right edge is cut at an angle.

Example 4: a blend-mode gradient overlay.

CSS

```css
.photo {
  isolation: isolate;
}
.photo::after {
  content: "";
  position: absolute;
  inset: 0;
  background: linear-gradient(90deg, #0969da, #8250df);
  mix-blend-mode: color;
}
```

Expected result: the photo is tinted with the gradient, and `isolation` stops the blend leaking to the page background.

15.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`filter` on a parent for a hover	blurs the text too	filter the child image
`backdrop-filter` without a translucent background	no visible effect	add `rgb(... / 0.x)`
Animating `filter: blur()`	repaints on every frame	animate `opacity`/`transform`
Blend modes without isolation	bleeds up the tree	`isolation: isolate`
`clip-path` with `overflow: hidden` surprises	double clipping	pick one
Heavy filters on mobile	battery and jank	test on a real device

- `@supports (backdrop-filter: blur(1px))` before relying on it.
- Keep decorative effects off interactive elements.

15.5 Practice exercises

1. Make an image black and white, in colour on hover.

2. Frost a panel over a photo.
3. Cut a 10px diagonal off the top-right corner of a card.
4. Which property keeps a blend mode inside a component?
5. Which two properties are cheapest to animate?

Solutions

1. `css img { filter: grayscale(1); transition: filter 200ms; } .card:hover img, .card:focus-within img { filter: none; }`
2. `css .panel { background: rgb(255 255 255 / 0.6); backdrop-filter: blur(10px); }`
3. `clip-path: polygon(0 0, calc(100% - 10px) 0, 100% 10px, 100% 100%, 0 100%);`
4. `isolation: isolate;`
5. `transform` and `opacity`.

15.6 Mini test

1. Multiple choice: which function blurs an element? - a) `blur()` - b) `soften()` - c) `haze()` - d) `vague()`
2. True or false: `backdrop-filter` affects the element itself.
3. Multiple choice: which property clips to a polygon? - a) `mask` - b) `clip-path` - c) `overflow` - d) `shape-outside`
4. Short code question: desaturate an image on hover.
5. Multiple choice: which blend mode darkens layers? - a) `screen` - b) `multiply` - c) `difference` - d) `overlay`
6. True or false: `filter` applies to descendants too.
7. Short answer: which property isolates blending?

Answer key and explanations

1. a - `blur(4px)`.
2. False - it affects the content behind the element.
3. b - `clip-path` with `polygon()`, `path()` or shape functions.
4. `img:hover { filter: saturate(0); }` (or set it on the parent card)
5. b - `multiply`.
6. True - it is applied to the element and everything it contains, so it also blurs text.
7. `isolation: isolate`.

Chapter 16 - Transitions, animations, view transitions

16.1 Summary

- A `transition` animates between two states on a property change.
- `transition: <property> <duration> <timing-function> <delay>`.
- `cubic-bezier()` and `steps()` give control over pacing; `ease-in-out` is a safe default.
- An `animation` needs `@keyframes` plus the animation properties.
- `animation-composition` and `animation-fill-mode` control final states.
- `transform` and `opacity` animate at 60fps; layout properties do not.
- `prefers-reduced-motion` must be honoured: shorten or remove motion.
- Scroll-driven animations (`animation-timeline`) replace scroll listeners.
- View transitions animate page and DOM changes with `view-transition-name`.
- Never animate a property just because you can: motion must mean something.

16.2 Concepts

Transitions

CSS

```css
transition: transform 150ms ease, box-shadow 150ms ease;
transition-property: transform;
transition-duration: 150ms;
transition-timing-function: cubic-bezier(0.4, 0, 0.2, 1);
transition-delay: 0ms;
```

Timing functions

Name	Feel
`linear`	constant speed (feels mechanical)
`ease`	quick start, slow end (default)
`ease-in`	accelerates
`ease-out`	decelerates (good for entrances)
`ease-in-out`	smooth both ends
`cubic-bezier(a, b, c, d)`	custom
`steps(4, jump-end)`	discrete steps

Animations

CSS

```css
@keyframes pulse {
  0%, 100% { transform: scale(1); }
  50% { transform: scale(1.05); }
}

.badge {
  animation: pulse 1.6s ease-in-out infinite;
  animation-fill-mode: both;
  animation-play-state: running;
}
```

Reduced motion

CSS

```css
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}
```

View transitions

CSS

```css
@view-transition { navigation: auto; }

:root { --card: hero; }
.card { view-transition-name: var(--card, card); }
```

With `@view-transition { navigation: auto; }` the browser cross-fades between pages, and elements sharing a name morph from their old position to the new one.

16.3 Examples

Example 1: a button that lifts on hover.

CSS

```css
.button {
  transform: translateY(0);
  box-shadow: 0 1px 2px rgb(0 0 0 / 0.1);
  transition: transform 120ms ease-out, box-shadow 120ms ease-out;
}
.button:hover {
  transform: translateY(-1px);
  box-shadow: 0 4px 12px rgb(0 0 0 / 0.15);
}
.button:active {
  transform: translateY(0);
  box-shadow: 0 1px 2px rgb(0 0 0 / 0.1);
}
```

Expected result: the button rises one pixel while hovered, presses back down on click, with a 120ms ease that reads as responsive rather than slow.

Example 2: a skeleton loading state.

CSS

```css
@keyframes shimmer {
  from { background-position: -200% 0; }
  to   { background-position: 200% 0; }
}

.skeleton {
  background: linear-gradient(90deg, #eaeef2 25%, #f6f8fa 37%, #eaeef2 63%);
  background-size: 200% 100%;
  animation: shimmer 1.4s linear infinite;
}
```

Expected result: a grey bar with a light band travelling left to right, looping while content loads.

Example 3: a step-based reveal.

CSS

```css
@keyframes rise {
  from { opacity: 0; transform: translateY(8px); }
  to   { opacity: 1; transform: none; }
}

.list > li {
  animation: rise 240ms ease-out both;
}
```

Expected result: each item fades in and slides up slightly; `both` keeps the start state before the animation begins and the end state after it.

Example 4: a spinner that respects reduced motion.

CSS

```css
.spinner {
  animation: spin 900ms linear infinite;
}
@keyframes spin { to { transform: rotate(1turn); } }

@media (prefers-reduced-motion: reduce) {
  .spinner { animation-duration: 2400ms; }
}
```

Expected result: normal users see a fast rotation; users who prefer less motion see a slow one, which still communicates "loading" without a spinning blur.

16.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Animating `width`/`height`/`top`	jank, layout thrash	`transform` and `opacity`
500ms+ transitions on hover	feels laggy	80-200ms
Motion for its own sake	distracting, accessibility issue	meaningful feedback

Mistake	Why it hurts	Best practice
Ignoring `prefers-reduced-motion`	WCAG failure	always handle it
`animation-fill-mode` forgotten	snaps back to the start	`both` or `forwards`
Animating `box-shadow`/`filter` on long lists	frame drops	animate a pseudo-element's opacity
Transitions on `display`	not animatable	use other state cues

- Use `transition-behavior: allow-discrete` for `display` fades in modern browsers.
- Keep total animation duration under 300ms for interactions.

16.5 Practice exercises

1. Fade a tooltip's opacity in 150ms.
2. Write keyframes for a pulse.
3. Animate an element 8px up on hover without triggering layout.
4. Disable animations for users who prefer reduced motion.
5. Which two properties should you animate for best performance?

Solutions

1. `css .tip { opacity: 0; transition: opacity 150ms ease; } .tip[data-open="true"] { opacity: 1; }`
2. `css @keyframes pulse { 0%, 100% { transform: scale(1); } 50% { transform: scale(1.05); } }`
3. `transform: translateY(-8px);` with `transition: transform 150ms ease;`
4. `css @media (prefers-reduced-motion: reduce) { *, *::before, *::after { animation: none !important; transition: none !important; } }`
5. `transform` and `opacity`.

16.6 Mini test

1. Multiple choice: how many values does the `transition` shorthand take at most? - a) two - b) three - c) four - d) six
2. True or false: `transition` animates between two states on the same element.
3. Multiple choice: which timing function eases out? - a) `ease-in` - b) `ease-out` - c) `linear` - d) `steps(2)`
4. Short code question: a 200ms colour transition.
5. Multiple choice: which property must accompany `@keyframes` on an element for an animation to run? - a) `animation` - b) `transition` - c) `transform` - d) `will-change`
6. True or false: animating `top` performs better than animating `transform`.
7. Short answer: which media feature tells you the user wants less motion?

Answer key and explanations

1. c - property, duration, timing function, delay.
2. True - it needs a start and an end value, triggered by a state change.
3. b - `ease-out`.
4. `transition: color 200ms ease;`
5. a - the `animation` shorthand (or its longhands).
6. False - `transform` is composited; `top` triggers layout on every frame.
7. `prefers-reduced-motion`.

Chapter 17 - Accessibility and performance

17.1 Summary

- Accessibility in CSS: contrast, focus order, target size, no motion, no colour-only meaning, readable type.
- `:focus-visible` styles are not optional.
- Respect `prefers-reduced-motion`, `prefers-contrast`, `forced-colors`.
- `forced-colors: active` requires system colours and explicit borders.
- Target size of at least 24x24 CSS pixels (WCAG 2.2) for pointers.
- Performance: minimise layout thrash, use `content-visibility`, avoid heavy filters and huge shadows.
- `will-change` is a hint, not a fix; remove it after use.
- Split critical CSS, load the rest asynchronously.
- Dark mode with `prefers-color-scheme` or `light-dark()`.
- A11y and performance are part of the CSS review, not a separate phase.

17.2 Concepts

Contrast and states

CSS

```css
:root {
  --accent: #0969da;         /* 4.6:1 on white */
  --danger: #cf222e;
  --ok: #1a7f37;
}

.error {
  color: var(--danger);
  border-inline-start: 4px solid var(--danger);
}
.error::before { content: "\26A0\FE0E  "; }   /* icon, not colour alone */
```

Forced colours and contrast preferences

CSS

```css
@media (forced-colors: active) {
  .card { border: 1px solid CanvasText; }
  .icon { forced-color-adjust: none; }
}

@media (prefers-contrast: more) {
  :root { --muted: #40464f; }
}
```

Deferred rendering

CSS

```css
.card--below {
  content-visibility: auto;
  contain-intrinsic-size: auto 320px;
}
```

The browser skips rendering off-screen content and keeps the scrollbar correct.

Critical CSS

HTML

```html
<link rel="stylesheet" href="critical.css">
<link rel="stylesheet" href="main.css" media="print" onload="this.media='all'">
```

Above-the-fold styles load first; the rest does not block rendering.

17.3 Examples

Example 1: an accessible disabled state.

CSS

```css
.button[disabled],
.button[aria-disabled="true"] {
  cursor: not-allowed;
  opacity: 0.6;
}
```

Expected result: the cursor communicates the state, and the button is visibly dimmed but still readable.

Example 2: a focus ring that works on any background.

CSS

```css
:focus-visible {
  outline: 2px solid var(--accent);
  outline-offset: 2px;
}
@media (forced-colors: active) {
  :focus-visible { outline-color: Highlight; }
}
```

Expected result: a blue ring normally; in forced-colours mode the operating system's highlight colour.

Example 3: off-screen content that still renders lazily.

CSS

```css
.feed > article {
  content-visibility: auto;
  contain-intrinsic-size: auto 400px;
}
```

Expected result: a long feed paints only the visible entries, so the initial render is fast and the scrollbar stays stable.

Example 4: safe animation defaults.

CSS

```css
:root { --dur: 160ms; }
@media (prefers-reduced-motion: reduce) { :root { --dur: 1ms; } }
.card { transition: transform var(--dur) ease; }
```

Expected result: every duration comes from one token, so reduced motion is a one-line change.

17.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`will-change` left on forever	wastes memory	toggle it around the animation
`content-visibility` on everything	can hurt search indexing and scroll	target long lists
Removing outlines for "clean" design	keyboard inaccessible	style `:focus-visible`
`opacity` alone for disabled states	unreadable text	also change cursor/colour
Colour-only error styling	WCAG failure	icon + text + colour
30+ expensive elements in view	low frame rate	simplify, animate composited props
Blocking stylesheet in `<head>` for a huge file	slow first paint	critical CSS + async rest

- Test with a keyboard only, at 200% zoom, and with reduced motion.

- Run Lighthouse on every page type you build.

17.5 Practice exercises

1. Write a `:focus-visible` ring.
2. Disable transitions when reduced motion is requested.
3. Which property lets the browser skip off-screen rendering?
4. Style a disabled button so it is clearly not clickable.
5. Why is `opacity` alone insufficient for a disabled state?

Solutions

1. `:focus-visible { outline: 2px solid var(--accent); outline-offset: 2px; }`
2. `css @media (prefers-reduced-motion: reduce) { * { transition: none !important; animation: none !important; } }`
3. `content-visibility: auto` (with `contain-intrinsic-size`).
4. `css .button[disabled] { cursor: not-allowed; opacity: 0.6; border-color: var(--border); }`
5. Because faded text can fall below the contrast minimum, and opacity changes nothing about role or cursor. Combine it with a cursor change, a colour change and `aria-disabled`.

17.6 Mini test

1. Multiple choice: which pseudo-class should style keyboard focus? - a) `:focus` - b) `:focus-visible` - c) `:focus-within` - d) `:target`
2. True or false: `content-visibility: auto` can improve performance.
3. Multiple choice: which media feature detects high-contrast mode? - a) `prefers-contrast: more` - b) `forced-colors: active` - c) `prefers-color-scheme: dark` - d) `contrast: 200%`
4. Short code question: respect reduced motion.
5. Multiple choice: which cursor marks a disabled control? - a) `default` - b) `not-allowed` - c) `wait` - d) `text`
6. True or false: `will-change` improves performance if left on permanently.
7. Short answer: name two WCAG requirements CSS can break.

Answer key and explanations

1. b - `:focus-visible`.
2. True - the browser skips rendering off-screen subtrees.
3. b - `forced-colors: active` (with `prefers-contrast` for the preference).
4. `css @media (prefers-reduced-motion: reduce) { * { animation: none !important; transition: none !important; } }`
5. b - `not-allowed`.
6. False - it reserves resources; leaving it on permanently wastes memory. Toggle it during animations only.

7. Any two of: contrast ratios, visible focus, target size, motion, colour-only meaning, text spacing.

Cheat sheet

18.1 Syntax

CSS

```css
selector { property: value; }
/* comment */
```

18.2 At-rules

`@media` `@supports` `@layer` `@container` `@property` `@font-face` `@import` `@keyframes` `@view-transition`

18.3 Selectors

Selector	Matches
`.c`	class
`#i`	id
`*`	everything
`[a="b"]`	attribute equals
`[a^=]` `[a$=]` `[a*=]`	starts / ends / contains
`a b` `a > b` `a + b` `a ~ b`	descendant / child / next / subsequent
`:hover` `:focus-visible` `:checked` `:disabled`	state
`:nth-child(2n)` `:nth-of-type(3)`	position
`:not(.x)` `:is(a, b)` `:where(a, b)` `:has(> img)`	logic
`::before` `::after` `::marker` `::placeholder` `::selection`	virtual

18.4 Specificity and cascade

`!important` > inline > id (1-0-0) > class/attr/pseudo-class (0-1-0) > type/pseudo-element (0-0-1). Later wins on a tie. `@layer` order beats all of it.

18.5 Box model

`box-sizing: border-box` | `content-box` | `margin` `padding` `border` `border-radius` | `inline-size` `block-size` | `min-width: 0` in flex/grid

18.6 Display

`block` `inline` `inline-block` `flex` `inline-flex` `grid` `inline-grid` `none` `contents` `flow-root`

18.7 Positioning

`static` `relative` `absolute` `fixed` `sticky` | `inset: 0` | `inset-inline-start/end` | `z-index` | `isolation: isolate`

18.8 Flexbox

`flex-direction` `flex-wrap` `gap` | `justify-content` (main axis) | `align-items` (cross axis) | `align-self` | `flex: 1 1 0` | `flex-shrink: 0` | `min-width: 0`

18.9 Grid

`grid-template-columns: repeat(auto-fill, minmax(200px, 1fr))` | `fr` | `1fr` | `gap` | `grid-template-areas` | `grid-column: span 2` | `grid-row: -1` | `place-items: center` | `subgrid`

18.10 Sizing

`px` `rem` `em` `%` `vw` `vh` `dvh` `svh` `ch` `cqi` | `min-content` `max-content` `fit-content` | `aspect-ratio: 16 / 9` | `clamp(min, val, max)` | `min()` `max()` `calc()`

18.11 Logical properties

`margin-inline-start/end` | `padding-block-start/end` | `inset-inline-start/end` | `inline-size` `block-size` | `text-align: start/end` | `border-inline-start`

18.12 Colour

`#rgb` `#rrggbbaa` | `rgb(r g b / a)` | `hsl(h s% l% / a)` | `oklch()` | `color-mix(in oklab, a, b)` | `currentColor` | `light-dark()`

18.13 Backgrounds

`background: url() center / cover no-repeat color` | `background-clip` `origin` | `linear-gradient()` `radial-gradient()` `conic-gradient()` | `repeating-linear-gradient()`

18.14 Typography

`font-family` `font-size` `line-height: 1.5` | `font-weight` | `letter-spacing` `word-spacing` | `text-wrap: balance | pretty` | `max-width: 70ch` | `font-variant-numeric: tabular-nums` | `text-transform`

18.15 Lists

`list-style: none` | `list-style-type` | `::marker` | `counter-reset` `counter-increment` `counter()`

18.16 Borders and shadows

`border: 2px solid color` | `border-radius` | `box-shadow` (two layers) | `box-shadow: inset` | `filter: drop-shadow()` |

18.17 Overflow and scroll

`overflow: visible | hidden | clip | scroll | auto` | `overscroll-behavior: contain` | `scroll-snap-type scroll-snap-align` | `scroll-margin-top` | `scroll-behavior: smooth` | `scrollbar-gutter: stable`

18.18 Effects

`filter: blur() brightness() contrast() grayscale() sepia() saturate() hue-rotate() drop-shadow()` | `backdrop-filter` | `mix-blend-mode` | `clip-path: circle() polygon() path()` | `mask-image` | `isolation: isolate`

18.19 Motion

`transition: prop duration timing delay` | `cubic-bezier(a,b,c,d) steps(n)` | `@keyframes` | `animation` | `animation-fill-mode: both` | `@view-transition { navigation: auto }` |

18.20 Accessibility and performance

`:focus-visible { outline: 2px solid; outline-offset: 2px }` | `@media (prefers-reduced-motion: reduce)` | `prefers-color-scheme` | `prefers-contrast` | `forced-colors: active` | `cursor: not-allowed` | `content-visibility: auto` | `contain-intrinsic-size` | `will-change` |

18.21 The values you type most

CSS

```css
*, *::before, *::after { box-sizing: border-box; }
body { font: 100%/1.5 system-ui, sans-serif; max-width: 70ch; }
.center { display: grid; place-items: center; }
.row { display: flex; gap: 1rem; align-items: center; }
.cards { display: grid; gap: 1rem;
  grid-template-columns: repeat(auto-fill, minmax(220px, 1fr)); }
```

Final test

1. Multiple choice: which `box-sizing` includes padding and border in the declared width? - a) `content-box` - b) `border-box` - c) `padding-box` - d) `margin-box`
2. True or false: adjacent vertical margins collapse to their sum.
3. Multiple choice: which selector matches keyboard focus? - a) `:focus` - b) `:focus-visible` - c) `:active` - d) `:hover`
4. Short code question: centre a child both ways with grid.
5. Multiple choice: which unit is relative to the root font size? - a) `em` - b) `rem` - c) `ch` - d) `vw`
6. Multiple choice: what does `flex: 1` expand to? - a) `1` - b) `1 1 0%` - c) `1 0 auto` - d) `initial`

7. True or false: Grid is one-dimensional.
8. Short code question: a responsive grid with 200px minimum tracks.
9. Multiple choice: which value of `position` keeps the element in normal flow? - a) `absolute` - b) `fixed` - c) `relative` - d) `sticky`
10. Multiple choice: which property adds space between flex items? - a) `margin` - b) `gap` - c) `padding` - d) `justify-content`
11. True or false: `::before` works without a `content` property.
12. Short code question: set a `--accent` token on `:root` and use it.
13. Multiple choice: which function clamps a value between bounds? - a) `min()` - b) `clamp()` - c) `max()` - d) `calc()`
14. Multiple choice: which property makes a component respond to its own container? - a) `@media` - b) `container-type` - c) `position` - d) `resize`
15. True or false: logical properties help with right-to-left languages.
16. Short code question: disable transitions for reduced-motion users.
17. Multiple choice: which property is cheapest to animate? - a) `height` - b) `transform` - c) `top` - d) `width`
18. Multiple choice: which property blurs what is behind an element? - a) `filter` - b) `backdrop-filter` - c) `mix-blend-mode` - d) `opacity`
19. Multiple choice: which unit is a fraction of leftover grid space? - a) `%` - b) `fr` - c) `em` - d) `vw`
20. True or false: `!important` is a good way to win specificity fights.
21. Short code question: a 1px border, 8px radius and a two-layer shadow.
22. Multiple choice: which `overflow` value clips without a scroll container? - a) `hidden` - b) `clip` - c) `scroll` - d) `auto`
23. Multiple choice: which keyword makes a custom property inherit the parent value? - a) `initial` - b) `inherit` - c) `unset` - d) `revert`
24. Multiple choice: which property styles the list marker? - a) `::before` - b) `::marker` - c) `list-style-type` only - d) `::bullet`
25. Short code question: `scroll-margin` for a 4rem sticky header.
26. Multiple choice: which at-rule declares a typed custom property? - a) `@property` - b) `@layer` - c) `@supports` - d) `@container`
27. True or false: `overflow-x: hidden` on `<html>` can break `position: sticky`.
28. Multiple choice: which value of `cursor` signals a blocked action? - a) `default` - b) `not-allowed` - c) `progress` - d) `text`
29. Multiple choice: which property limits line length? - a) `max-width: 70ch` - b) `width: 70ch` - c) `overflow: hidden` - d) `text-wrap: balance`
30. Short code question: a mobile-first media query for two columns at 40em.

19.1 Final test answers

1. b - `border-box`.
2. False - adjacent vertical margins collapse to the larger value.

3. b - `:focus-visible`.
4. `display: grid; place-items: center;`
5. b - `rem`.
6. b - `1 1 0%`.
7. False - Grid is two-dimensional; Flexbox is one-dimensional.
8. `grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));`
9. c - `relative` offsets without leaving the flow.
10. b - `gap` (also `row-gap`/`column-gap`).
11. False - `content` is required.
12. `css :root { --accent: #0969da; } a { color: var(--accent); }`
13. b - `clamp(min, preferred, max)`.
14. b - `container-type` (queried with `@container`).
15. True - `margin-inline-start` etc. flip automatically with `direction`.
16. `css @media (prefers-reduced-motion: reduce) { * { transition: none !important; animation: none !important; } }`
17. b - `transform` (with `opacity`) is composited.
18. b - `backdrop-filter`.
19. b - `fr`.
20. False - use `@layer` or fix the specificity.
21. `css border: 1px solid var(--border); border-radius: 8px; box-shadow: 0 1px 2px rgb(0 0 0 / 0.06), 0 4px 12px rgb(0 0 0 / 0.08);`
22. b - `clip`.
23. b - `inherit`.
24. b - `::marker`.
25. `scroll-margin-top: 4rem;`
26. a - `@property`.
27. True - `html` stops being the scroll container, so descendants have no reference to stick to.
28. b - `not-allowed`.
29. a - `max-width: 70ch`.
30. `css .cards { display: grid; gap: 1rem; } @media (min-width: 40em) { .cards { grid-template-columns: repeat(2, 1fr); } }`

Sources

Every URL below was consulted while planning and writing this volume. No text, images or code blocks were copied from these pages; all explanations, examples, exercises, tests and answers were written from scratch.

- https://web.dev/learn/css

- https://web.dev/learn/css/welcome
- https://web.dev/learn/css/box-model
- https://web.dev/learn/css/selectors
- https://web.dev/learn/css/nesting
- https://web.dev/learn/css/the-cascade
- https://web.dev/learn/css/specificity
- https://web.dev/learn/css/inheritance
- https://web.dev/learn/css/color
- https://web.dev/learn/css/sizing
- https://web.dev/learn/css/layout
- https://web.dev/learn/css/flexbox
- https://web.dev/learn/css/grid
- https://web.dev/learn/css/logical-properties
- https://web.dev/learn/css/custom-properties
- https://web.dev/learn/css/spacing
- https://web.dev/learn/css/pseudo-elements
- https://web.dev/learn/css/pseudo-classes
- https://web.dev/learn/css/borders
- https://web.dev/learn/css/shadows
- https://web.dev/learn/css/focus
- https://web.dev/learn/css/cursors-and-pointers
- https://web.dev/learn/css/z-index
- https://web.dev/learn/css/anchor-positioning
- https://web.dev/learn/css/popover-and-dialog
- https://web.dev/learn/css/functions
- https://web.dev/learn/css/paths-shapes-clipping-masking
- https://web.dev/learn/css/gradients
- https://web.dev/learn/css/animations
- https://web.dev/learn/css/filters
- https://web.dev/learn/css/blend-modes
- https://web.dev/learn/css/lists
- https://web.dev/learn/css/counters
- https://web.dev/learn/css/transitions
- https://web.dev/learn/css/view-transitions-spas
- https://web.dev/learn/css/overflow
- https://web.dev/learn/css/backgrounds
- https://web.dev/learn/css/typography
- https://web.dev/learn/css/container-queries

- https://web.dev/learn/css/conclusion
- https://web.dev/learn/css/quiz
- https://developer.mozilla.org/en-US/docs/Web/CSS - used to check property syntax, initial values and browser support while writing the examples